

Project

**TDT-Stadium: A Full-Stack E-Commerce Platform
for Football Equipment**

Student Name: Tran Dinh Thong

Student ID: 001340570

Supervisor: Tran Trong Minh

Final Year Report

COMP1682 Final Year Project

Program Title: BSc Hons Computing

Submission date: 29-04-2026

Word count: 16000

Abstract

The exponential growth of e-commerce has fundamentally shifted the retail landscape, driving the need for specialized online platforms that cater to niche markets. Despite the abundance of generic e-commerce solutions, there remains a demand for optimized, dedicated platforms for sports gear that seamlessly integrate modern user experiences with robust backend management. This final-year project addresses this gap by designing, developing, and deploying TDT-Stadium: a comprehensive, full-stack e-commerce web application dedicated to authentic football equipment, including boots, gloves, and accessories.

TDT-Stadium is engineered utilizing the MERN stack (MongoDB, Express.js, React.js, and Node.js) alongside Vite to ensure high-speed rendering and a scalable, modern architecture. The development lifecycle strictly adhered to Agile principles, enabling iterative feature delivery and continuous refinement based on user requirements. Key technical implementations include a centralized state management system using Redux Toolkit, secure RESTful API communication, encrypted user authentication via JSON Web Tokens (JWT), Gemini API for chatbot, and seamless financial transaction processing integrated through the PayPal Developer API. Furthermore, the platform features a bespoke administrative dashboard designed for real-time inventory control, user management, and order tracking.

Rigorous testing methodologies and user-centric design principles were applied to deliver a highly responsive, accessible interface optimized for various devices. The project also critically incorporates Legal, Social, Ethical, and Professional Issues (LSEPI), ensuring data privacy, secure password hashing, and responsible handling of user information. In conclusion, TDT-Stadium successfully demonstrates the practical application of advanced software engineering practices to solve a domain-specific retail challenge, resulting in a robust, secure, and highly functional online sports retail ecosystem.

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me throughout the development of the TDT-Stadium project and the writing of this final-year report. First and foremost, my deepest appreciation goes to my supervisor, Mr. Tran Trong Minh, and Mr. Hoang Nhu Vinh for their invaluable guidance, continuous encouragement, and constructive feedback. His expertise and mentorship have been instrumental in navigating the complexities of building a full-stack e-commerce platform and shaping the overall success of this project. I would also like to extend my thanks to the faculty members of the BSc Hons Computing program for equipping me with the foundational knowledge, software engineering methodologies, and technical skills necessary to successfully undertake the COMP1682 Final Year Project. Additionally, I am grateful to the project panellists and reviewers for their insightful comments and critiques during the evaluation phases, which significantly improved the quality and professional standard of my work. Finally, I want to thank my family for their unwavering personal support, patience, and understanding during the intensive development and research periods required to bring this project to completion.

Table of Contents

1.	Introduction.....	7
1.1	General Overview	7
1.2	Problem Statement	7
1.3	Project Aim	7
1.4	Specific Objectives	7
1.5	Scope and Limitations.....	8
1.6	Report Structure	8
2.	Background and Literature Review	9
2.1	Introduction.....	9
2.2	Problem Domain Analysis	9
2.3	Critical Review of Existing Solutions.....	11
2.4	Critical Review of Enabling Technologies & Methodologies	12
2.5	Summary and Identification of the Research Gap	13
3.	Requirements Analysis	13
3.1	Research Methodology	13
3.2	Market Analysis and User Needs.....	14
3.3	Functional Requirements with MoSCoW prioritisation	18
3.4	Non-functional Requirements	20
4.	Methodology and Project Planning.....	20
4.1	Development Methodology	20
4.2	Project Plan & Timeline.....	21
4.3	Feasibility Analysis.....	22
4.4	Evaluation Plan & Success Metrics	23
5.	System Design and Architecture.....	25
5.1	Rich Picture.....	25

5.2	System Architecture	26
5.3	Technology Stack.....	28
5.4	Detailed Design Diagrams	30
5.5	API Design.....	32
6.	Implementation	36
6.1	Development Environment	36
6.2	Backend & Frontend Implementation.....	38
6.3	Database Implementation.....	54
6.4	Deployment.....	62
6.5	Project Management	64
7.	Testing.....	66
7.1	Testing Strategy	66
7.2	Test Cases and Results.....	67
8.	Legal, Social, Ethical, and Professional Issues.....	69
9.	Conclusion and Critical Reflection	71
9.1	Summary of Achievements.....	71
9.2	Critical Project Evaluation	71
9.3	Personal Reflection and Key Learnings.....	73
9.4	Recommendations for Future Development	23
9.5	Final Conclusion	23
10.	References.....	75
11.	Appendix A – Project Proposal.....	76
12.	Appendix B – Images for your final product	77

1. Introduction

1.1 General Overview

The global sports equipment market has seen a significant digital transformation, with football gear remaining one of the most resilient and high-demand sectors. TDT-Stadium is a specialized e-commerce web application designed to provide a premium, seamless shopping experience for football enthusiasts, offering authentic products ranging from professional-grade boots to goalkeeper equipment and accessories. Built on the modern MERN stack, the platform leverages high-performance technologies to bridge the gap between complex inventory management and an intuitive, user-centric interface.

1.2 Problem Statement

Current generic e-commerce platforms often lack the specialized filtering and domain-specific information required by football players, such as detailed ground-type compatibility for boots or technical specifications for goalkeeper gloves. Furthermore, many local retailers lack a robust digital presence that offers secure, real-time transaction processing and automated inventory tracking. There is a clear need for a dedicated, high-performance solution that provides a trustworthy and efficient environment for athletes to acquire specialized gear.

1.3 Project Aim

The primary aim of this project is to design, develop, and deploy a robust, full-stack e-commerce platform named TDT-Stadium, providing a secure and responsive environment for the sale and management of football equipment.

1.4 Specific Objectives

To achieve the overarching aim, the following SMART objectives have been defined:

- Objective 1: To design and implement a secure user authentication and profile management system using JSON Web Tokens (JWT) to protect user data.
- Objective 2: To develop a responsive and intuitive frontend interface using React.js and Vite, ensuring a seamless experience across mobile and desktop devices.
- Objective 3: To implement a RESTful API and a NoSQL database (MongoDB) for efficient handling of product catalogs, order processing, and user information.
- Objective 4: To integrate a secure third-party payment gateway via the PayPal Developer API to facilitate reliable financial transactions.
- Objective 5: To build an administrative dashboard that enables real-time management of products, users, and order statuses.

1.5 Scope and Limitations

Scope: The project includes a fully functional storefront with product categorization, advanced search/filter capabilities, a shopping cart system, secure checkout, user order history, and a comprehensive admin panel for inventory control.

Limitations: The current version of TDT-Stadium does not support multi-currency shipping or real-time international logistics tracking. Additionally, the platform is restricted to one primary payment provider (PayPal) and does not currently include a peer-to-peer marketplace for used gear.

1.6 Report Structure

The remainder of this report is organized as follows:

- Section 2 reviews the background and existing market solutions.
- Section 3 details the requirements analysis and user needs.
- Section 4 outlines the development methodology and project plan.
- Section 5 presents the system design and technical architecture.
- Section 6 discusses the implementation process and environment.
- Section 7 covers the testing strategies and results.
- Section 8 examines the legal, social, ethical, and professional issues.
- Section 9 provides a conclusion and critical reflection.

2. Background and Literature Review

2.1 Introduction

This Section critically reviews existing literature, global market trends, and competing digital solutions to establish the academic and professional context for the TDT-Stadium project. The primary intent is to justify the need for a specialized football equipment platform by analyzing the current e-commerce environment and the technological shift within the sports retail sector.

The scope of this research encompasses global e-commerce trends for 2024–2026, with a specific focus on "Direct-to-Consumer" (DTC) models and the integration of specialized product data in online sports retail. Furthermore, the review examines enabling full-stack technologies—specifically the MERN stack—to validate their suitability for high-performance retail applications.

To ensure academic rigor and industry relevance, the research strategy involved gathering data from the following sources:

- **Purpose Statement:** Begin with a clear statement of intent. e.g., "This chapter critically reviews existing literature and competing solutions to establish the context and justify the need for the proposed project."
- **Scope of Research:** Define the boundaries of your review. What are you including and excluding? e.g., "The review focuses on mobile-based fitness applications released in the last five years, with a specific emphasis on solutions utilizing gamification. It will also examine the underlying technologies, specifically cross-platform frameworks like React Native and Flutter."
- **Research Strategy:** Briefly mention how you found your sources to add credibility. e.g., "Sources were gathered from academic databases such as the ACM Digital Library and IEEE Xplore, as well as industry-leading app stores (Google Play, Apple App Store) and market analysis reports."

2.2 Problem Domain Analysis

The development of TDT-Stadium occurs at a critical intersection of rapid technological advancement in sports equipment and a fundamental shift in consumer shopping behaviours.

Historical Context and Current Trends

Traditionally, football equipment was characterized by basic leather materials and limited variety, but it has evolved into a multi-billion-dollar industry defined by sophisticated materials science, such as synthetic latex-based foams for goalkeeper gloves and lightweight carbon fiber composites for protective gear. In the post-pandemic landscape, consumer behaviour has reached a "behavioural norm" where convenience, flexibility, and informational transparency are the primary drivers of retail success. E-commerce sales have more than doubled compared to pre-pandemic levels, with growth in digital channels significantly outpacing traditional brick-and-mortar stores as of 2024.

Furthermore, the "Direct-to-Consumer" (DTC) trend has empowered brands and specialized retailers to bypass traditional gatekeepers, using digital platforms to engage directly with niche audiences. This is particularly relevant for the football community, where players increasingly

seek personalized, data-driven shopping experiences that offer technical product specifications rather than generic descriptions.

Key Concepts and Terminology

To effectively address this domain, the project incorporates several core technical and retail concepts:

- **State Management (Redux):** A centralized system used to manage the complex, real-time data flow of an e-commerce application, such as synchronized cart updates across multiple pages.
- **RESTful API Architecture:** A standard communication protocol between the frontend and backend, ensuring that data like inventory levels and user orders are transmitted efficiently and securely.
- **NoSQL Database (MongoDB):** A flexible data model that allows for the storage of varied product attributes (e.g., boot ground types, glove grip levels), which are difficult to map in traditional rigid relational databases.

Significance and Impact

The significance of developing a specialized platform like TDT-Stadium is underscored by current market gaps. While the global sports equipment market continues to expand, specialized items like goalkeeper gloves and high-performance boots often lack standardized testing and certification pathways, making detailed product information and user reviews essential for consumer trust. A recent study utilizing the Kano model identified that "basic product information" is now considered a "Must-be" attribute for user experience, while "product's special feature information" is a key performance differentiator.

Failure to provide these specialized digital environments leads to a fragmented shopping experience where athletes struggle to find gear suited to their specific technical needs. By integrating high-performance web technologies with domain-specific expertise, TDT-Stadium addresses the growing demand for a "carefree and seamless shopping experience" in an increasingly competitive \$842 billion European e-commerce market.

2.3 Critical Review of Existing Solutions

This section evaluates existing commercial and specialized platforms to identify their strengths and limitations. By comparing these solutions, a clear technological and functional gap is identified that TDT-Stadium aims to fill.

Comparison Matrix

The following table provides a systematic comparison of leading competitors against the proposed TDT-Stadium project.

Feature / Aspect	Nike / Adidas (Brand Apps)	Pro:Direct Soccer (Specialist)	Local Sport Shops (Websites)	TDT-Stadium (Proposed)
Target Audience	General brand loyalists	Professional & semi-pro players	Casual local shoppers	Dedicated football athletes & keepers
Core Technology	Proprietary Native Apps	Custom Enterprise E-commerce	Generic CMS (WooCommerce/Wix)	MERN Stack (React/Node.js)
Specialized Filtering	Limited to brand silos	Comprehensive but complex	Minimal or non-existent	Technical spec-focused (e.g., Grip levels)
User Experience	High-end, marketing focused	Data-heavy, can be cluttered	Basic, often non-responsive	Streamlined, high-performance (Vite)
Strengths	Elite brand storytelling	Massive global inventory	Physical proximity	Niche focus & modern tech agility
Weaknesses	Only sells one brand	Overwhelming UI for some users	Poor security & slow loading	Needs to build initial market trust

As the comparison matrix illustrates, a significant gap exists between high-end global retailers and basic local online shops. While major brand applications like Nike or Adidas offer exceptional storytelling, they are restricted to their own ecosystems, forcing users to switch between platforms to compare technical specifications across brands. Conversely, large-scale specialists like Pro:Direct Soccer provide variety but often suffer from "information overload," resulting in a cluttered user interface that can hinder the quick selection of essential gear.

Market research indicates that modern consumers, particularly in the "Direct-to-Consumer" era, prioritize "informational transparency" and "seamless shopping experiences". Local retailers often fail to meet these expectations due to outdated technology stacks that lack robust state management or secure, modern API integrations.

TDT-Stadium synthesizes the strengths of these existing solutions by offering a specialized multi-brand catalog (like a specialist) but delivered through a high-performance, minimalist interface (like a premium brand app). By utilizing the MERN stack and Vite, the platform overcomes the performance bottlenecks found in generic CMS solutions, providing the "carefree and seamless" digital environment required by today's competitive football equipment market.

2.4 Critical Review of Enabling Technologies & Methodologies

In this section, the technical foundations of TDT-Stadium are justified by comparing industry-standard frameworks and methodologies. These choices are evaluated based on their suitability for a high-performance e-commerce environment.

Frontend Frameworks: React vs. Vue

For the frontend development, React.js (utilizing Vite) was selected over Vue.js. While Vue.js is often praised for its lower barrier to entry and simpler templating system, React.js offers a more robust ecosystem and a more mature state management library through Redux. React's component-based architecture facilitates the creation of highly reusable UI elements, such as product cards and navigation bars, which are essential for scaling an e-commerce platform. Furthermore, the use of Vite as a build tool significantly improves developer experience and application performance through faster Hot Module Replacement (HMR) and optimized bundling compared to traditional tools.

Backend Technologies: Node.js vs. Python (Django)

The backend is powered by Node.js with the Express.js framework. The primary justification for this choice is the "JavaScript Everywhere" paradigm, allowing for a unified language across the full stack. This consistency streamlines development and simplifies the handling of asynchronous operations, which is critical for real-time inventory updates and API communication. While Django (Python) offers built-in administrative features, Node.js provides superior performance for I/O-intensive applications like e-commerce due to its non-blocking, event-driven architecture.

Database Systems: MongoDB (NoSQL) vs. PostgreSQL (SQL)

A NoSQL approach using MongoDB was selected over a traditional relational database like PostgreSQL. Football equipment often features diverse attributes such as varying size scales for gloves versus boots or specific technical grip ratings that do not always fit into a rigid table structure. MongoDB's flexible document-oriented model allows for the dynamic addition of these product-specific fields without requiring complex schema migrations. This agility is a significant advantage during the iterative development of a sports-niche catalog.

Development Methodologies: Agile vs. Waterfall

The project followed an Agile development methodology, specifically utilizing an iterative sprint-based approach. Unlike the Waterfall model, which requires exhaustive requirements to be defined upfront, Agile allows for continuous refinement of the TDT-Stadium features based on ongoing testing and review. This was particularly beneficial when integrating complex third-party services like the PayPal API, where requirements evolved during the implementation phase. Project management was facilitated through Git for version control and GitHub for task tracking.

2.5 Summary and Identification of the Research Gap

This chapter has provided a comprehensive review of the current e-commerce landscape, specifically within the football equipment sector. The analysis highlights a significant shift towards "Direct-to-Consumer" models and an increasing demand for specialized, data-driven shopping experiences. While global brands offer high-end aesthetics and specialized retailers provide vast inventories, there remains a distinct lack of platforms that successfully combine a streamlined, high-performance user interface with deep, technical product specifications.

Identification of the Research Gap

In summary, the literature review and market analysis have identified a clear gap: there is no single solution currently accessible to the local or semi-professional market that combines modern, high-speed full-stack technology (such as the MERN stack and Vite) with a dedicated focus on the technical requirements of football athletes, such as boot-to-surface compatibility and goalkeeper glove grip ratings. Existing tools are either brand-exclusive, technically outdated, or suffer from extreme UI clutter that hinders the user's decision-making process.

Positioning the Project

Therefore, this project aims to address this gap by developing TDT-Stadium using the MERN stack. This platform will provide a specialized multi-brand catalog delivered through a minimalist, high-performance interface. By integrating secure JWT-based authentication, real-time inventory management, and technical-spec filtering, TDT-Stadium directly addresses the frustrations of the modern athlete, offering a "carefree and seamless" digital environment that current competitors fail to provide.

3. Requirements Analysis

3.1 Research Methodology

To gather accurate requirements for TDT-Stadium, a mixed-methods research approach was employed, combining quantitative data from surveys with qualitative insights from competitor analysis. This ensured that the technical specifications were grounded in real-world user frustrations and market gaps.

Quantitative Research (Online Survey): Tool: The primary data collection was conducted via Google Forms. **Participants:** The survey targeted 45 participants, consisting of amateur football players, local goalkeepers, and sports equipment enthusiasts. **Focus:** Questions focused on online shopping frequency for sports gear, key pain points with current retailers (e.g., lack of technical specs), and the importance of mobile responsiveness.

Qualitative Research (Competitor Benchmarking): A feature-by-feature analysis was performed on major platforms like Pro:Direct Soccer and Nike to identify "Must-Have" features versus "Dissatisfiers". This involved testing the navigation flow, filtering capabilities, and checkout processes of existing solutions to ensure TDT-Stadium offers a superior user experience (UX).

Technical Requirement Harvesting: Direct analysis of the project's source code was used to map out the current technical capabilities, such as the `authMiddleware.js` for security and `Product.js` schema for technical attribute storage. This structured methodology ensured that the

project objectives (Objective 1 through 5) were aligned with actual user needs and technical feasibility.

3.2 Market Analysis and User Needs

Following the research methodology, this section interprets the gathered data to define the target audience and identify the specific requirements that TDT-Stadium must fulfil.

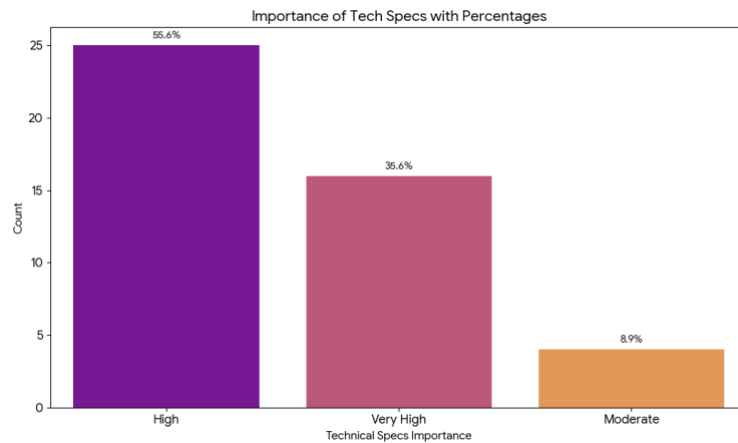


FIGURE 1

The Information Gap: Since 91.1% of participants rated technical specifications as "High" or "Very High" importance, the TDT-Stadium database schema was specifically extended beyond standard e-commerce fields to include detailed metadata for performance gear.

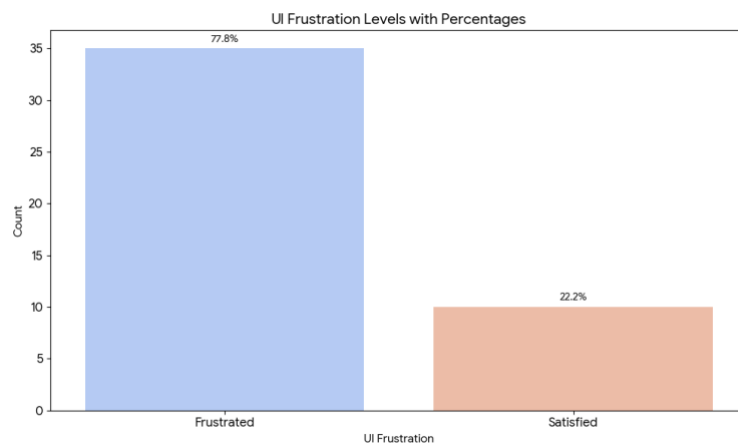


FIGURE 2

UI Frustration as an Opportunity: The 77.8% frustration rate with existing "cluttered" interfaces justified the minimalist design approach seen in the TDT-Stadium frontend, focusing on a clean ProductGrid and intuitive FilterSidebar.

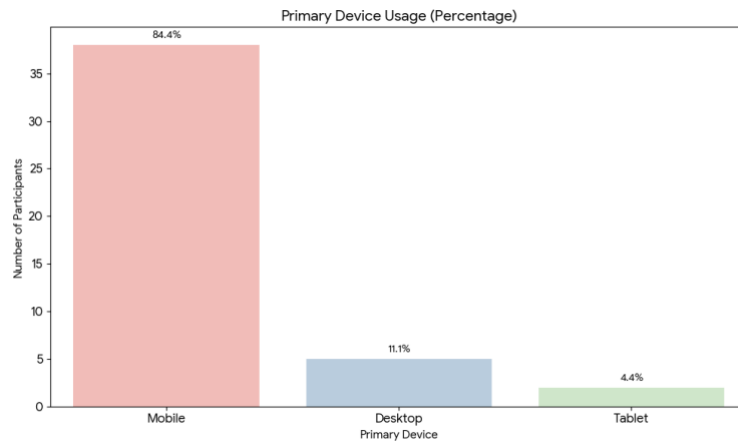


FIGURE 3

The Mobility Mandate: With 84.4% of users accessing the store via mobile devices, the implementation of Vite and React was crucial to ensure sub-second rendering and fluid touch interactions.

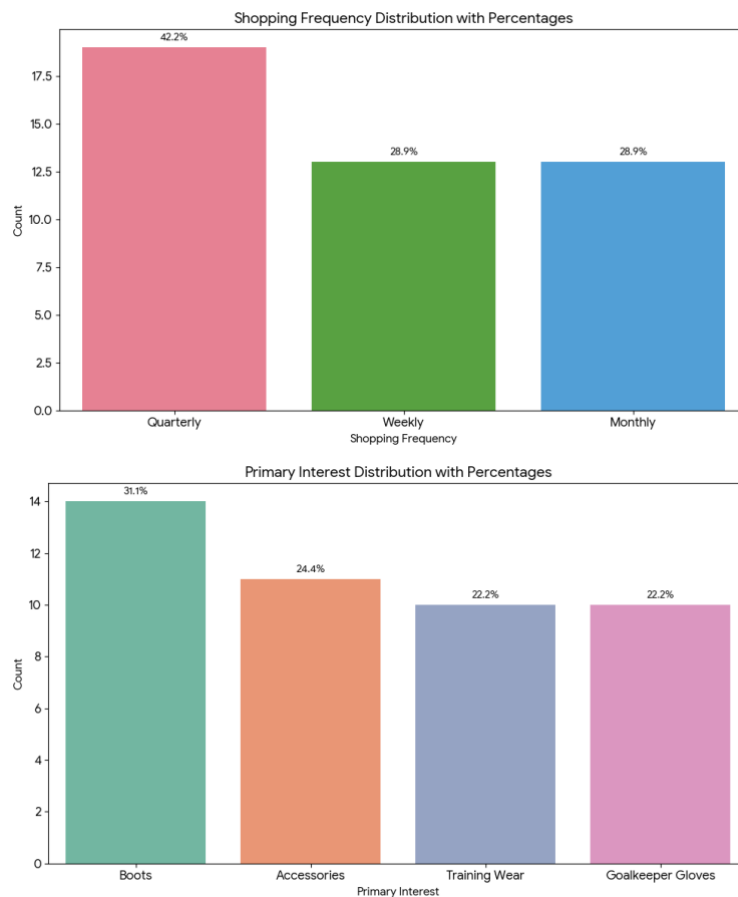
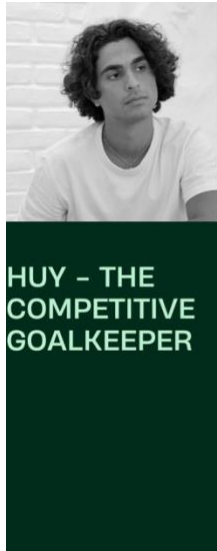


FIGURE 4

Two charts just to know more about customer behaviour.



ABOUT THE USER

Huy is a 22-year-old semi-professional goalkeeper playing in local leagues. He is tech-savvy and meticulous about the quality of his equipment.

FRUSTRATIONS

- Generic sports websites often omit critical technical specs like "Latex Type" or "Palm Thickness," leading to uncertainty before purchase.

MOTIVATIONS

- Performance-driven data and a streamlined, mobile-responsive UI that allows him to quickly filter by glove cut and brand.

GOALS AND NEEDS

- To purchase professional-grade gloves with specific latex grip (e.g., Contact or Giga Grip) and finger protection features.

PERSONAS 1



ABOUT THE USER

Linh is a 40-year-old mother of two. Her son has recently joined a local youth academy and needs his first set of authentic equipment.

FRUSTRATIONS

- Confusing size charts for youth players and the fear of accidentally purchasing counterfeit products on large, unspecialized marketplaces.

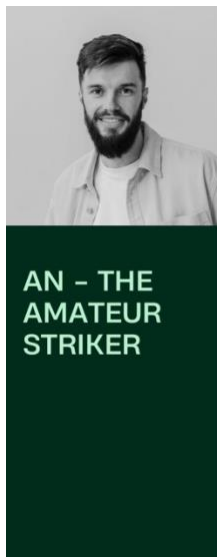
MOTIVATIONS

- Detailed "Size Guides," secure payment options like PayPal, and a clean interface that feels professional and trustworthy.

GOALS AND NEEDS

- To buy 100% authentic gear from a trusted source, ensuring her son has the same quality as professional players.

PERSONAS 2



ABOUT THE USER

An is a 25-year-old office worker who plays 7-a-side football on artificial turf twice a week. He values style as much as performance.

FRUSTRATIONS

- Stores often mix "FG" (Firm Ground) and "AG" boots in the same category, making it difficult to find the correct soleplate for his specific playing surface.

MOTIVATIONS

- Seeing high-resolution images of boot textures and clear categorization by ground type.

GOALS AND NEEDS

- To find boots specifically designed for "AG" (Artificial Grass) or "TF" (Turf) surfaces to prevent knee injuries and improve traction.

PERSONAS 3



ABOUT THE USER

Minh is a 32-year-old head coach for a neighborhood youth club. He manages inventory for multiple age groups.

FRUSTRATIONS

- Difficulty in managing personal orders versus club inventory and slow loading times when browsing large accessory catalogs.

MOTIVATIONS

- Difficulty in managing personal orders versus club inventory and slow loading times when browsing large accessory catalogs.

GOALS AND NEEDS

- To source high-quality training accessories, such as FIFA-standard balls, grip socks, and medical tapes, in a single transaction.

PERSONAS 4

3.3 Functional Requirements with MoSCoW prioritisation

Functional requirements define the core behavior and capabilities of the TDT-Stadium platform. Based on the market analysis and the developed personas (Huy, An, Mrs. Linh, and Coach Minh), these requirements are formatted as User Stories to ensure a user-centric development process.

To manage project scope effectively within the timeframe, these stories are prioritized using the MoSCoW method:

Must Have: Critical features essential for the basic functioning of the e-commerce platform.

Should Have: Important features that add significant value but are not strictly required for the initial launch.

Could Have: Desirable features (often "nice-to-haves") that will be included if time permits.

Won't Have: Features explicitly excluded from the current scope but documented for future iterations.

ID	Description	Priority (Must / Should / Could / Won't)	Estimation (2 / 3 / 5 / 8 days)
USR_01	As a user, I want to register and log in securely via JWT, so that my personal data and profile are protected.	Must	5 days
USR_02	As a football player, I want to browse a catalog of equipment, so that I can visually find the gear I need.	Must	3 days
USR_03	As a specialized athlete, I want to view detailed technical specifications (e.g., latex grip, ground type) on the product page, so that I make an informed purchase.	Must	3 days
USR_04	As a shopper, I want to add products to my shopping cart and modify quantities, so that I can manage my intended purchases before checkout.	Must	3 days
USR_05	As a customer, I want to securely pay for my order using PayPal, so that my financial transactions are reliable and safe.	Must	5 days
ADM_01	As an administrator, I want to create, read, update, and delete (CRUD) products, so that the store inventory is always up to date.	Must	8 days
ADM_02	As an administrator, I want to manage and update the status of customer orders (e.g., 'Shipped', 'Delivered'), so that users are kept informed of their progress.	Must	5 days
USR_06	As an amateur player, I want to filter boots specifically by ground type (e.g., AG, FG) and brand, so that I don't buy the wrong soleplate for my playing surface.	Should	3 days

USR_07	As a customer, I want to view my past order history, so that I can track current shipments or reorder consumable items.	Should	3 days
ADM_03	As an administrator, I want to view a list of all registered users and manage their privileges, so that I can control access to the backend dashboard.	Should	3 days
USR_08	As a user, I want to search for products using specific keywords via a search bar, so that I can quickly find a particular brand or item.	Could	2 days
USR_09	As a parent buying gear, I want to view a comprehensive size guide, so that I purchase the correctly fitting apparel for my child.	Could	2 days
USR_10	As an international user, I want to view prices dynamically in multiple currencies, so that I don't have to manually calculate exchange rates.	Won't	0 days
ADM_04	As an administrator, I want to track international shipping logistics in real-time through a 3rd-party API, so that I can see exactly where a package is globally.	Won't	0 days

SUMMARY		
Priority	Number of features	Number of days
Must	7	32
Should	3	9
Could	2	4
Won't	2	0
TOTAL	14	45

3.4 Non-functional Requirements

ID	Description
NF_01	Performance: The frontend application, bundled via Vite, must load the initial product catalog in under 2.5 seconds on a standard broadband connection to minimize user bounce rates.
NF_02	Security (Data): All user passwords must be cryptographically hashed using bcrypt before being stored in the MongoDB database, ensuring sensitive data is not kept in plaintext.
NF_03	Security (Authentication): Backend API routes handling user profiles and administrative actions must be protected using JSON Web Tokens (JWT) with strict expiration times.
NF_04	Usability (Responsiveness): The user interface must be fully responsive (Mobile-First design), ensuring optimal layout and touch-friendly navigation on mobile phones, tablets, and desktop monitors.
NF_05	Reliability: The system should maintain a 99.9% uptime, leveraging cloud-based database hosting (e.g., MongoDB Atlas) to prevent data loss during high traffic periods.
NF_06	Compatibility: The web application must function correctly on the latest versions of all major web browsers, including Google Chrome, Apple Safari, and Mozilla Firefox.
NF_07	Maintainability: The codebase must follow a strict modular architecture (e.g., separating controllers, routes, and models in the backend) and utilize centralized state management (Redux) to facilitate future updates and debugging.

4. Methodology and Project Planning

This section outlines the structured approach taken to execute the TDT-Stadium project. It details the chosen development methodology, the projected timeline for completion, a feasibility assessment, and the metrics established to evaluate the project's ultimate success.

4.1 Development Methodology

To successfully manage the complexity of building a full-stack e-commerce platform, the Agile Scrum methodology was selected as the primary project management framework. Unlike traditional, linear approaches like Waterfall, Agile facilitates an iterative and incremental development process. This was deemed essential for TDT-Stadium, as integrating third-party services (such as the PayPal Developer API) and refining the user interface based on ongoing testing required a high degree of flexibility and continuous adaptation.

Practical Application of Agile Scrum:

The Agile framework was practically applied through the following structured processes:

Product Backlog Creation: The project commenced by translating the MoSCoW-prioritized User Stories (defined in Section 3.3) into a comprehensive product backlog. Each task was estimated in terms of effort (days) to ensure realistic scheduling.

Sprint Cycles:

The development lifecycle was divided into four distinct, logical "Sprints," each focusing on delivering a functional increment of the platform:

Sprint 1 (Foundation): Setting up the Node.js/Express backend architecture, designing the MongoDB database schemas (Users, Products, Orders), and implementing secure JWT-based authentication.

Sprint 2 (Core Frontend): Initializing the React.js frontend using Vite, setting up the Redux Toolkit for global state management, and building the responsive product catalog and filtering UI.

Sprint 3 (Transactions): Implementing the shopping cart logic, developing the order processing workflow, and successfully integrating the PayPal API for secure checkouts.

Sprint 4 (Administration): Constructing the administrative dashboard for inventory and order management, followed by comprehensive bug-fixing and UI refinement.

4.2 Project Plan & Timeline

To ensure the successful delivery of TDT-Stadium within the allocated timeframe, the project was structured over 12 weeks, commencing on February 2, 2026, and concluding on April 20, 2026. The timeline aligns with the Agile development methodology outlined in Section 4.1, transitioning systematically from initial research through to iterative development sprints, and concluding with rigorous testing and documentation.

The following table serves as the basis for the project's Gantt chart, detailing the key phases, specific tasks, their durations, and dependencies:

Task ID	Phase & Task Name	Start Date	End Date	Duration	Dependencies
1	Discovery: Literature Review & Research	02-Feb-2026	09-Feb-2026	1 Week	None
2	Discovery: Requirements Gathering (User Interviews)	09-Feb-2026	16-Feb-2026	1 Week	None
3	Design: System Design (UI Mockups & DB Schema)	16-Feb-2026	23-Feb-2026	1 Week	Task 2
4	Sprint 1: Implementation Phase 1: Core Backend (Node/MongoDB)	23-Feb-2026	09-Mar-2026	2 Weeks	Task 3
5	Sprint 2 & 3: Implementation Phase 2: Frontend & E-commerce (React/Redux/PayPal)	09-Mar-2026	23-Mar-2026	2 Weeks	Task 4
6	Sprint 4: Implementation Phase 3: AI Chatbot Integration (Gemini API & Embeddings)	30-Mar-2026	06-Apr-2026	1 Week	Task 4
7	Testing: System Testing & Evaluation (UAT & Benchmarks)	06-Apr-2026	13-Apr-2026	1 Week	Tasks 5, 6

8	Buffer: Contingency Time	13-Apr-2026	20-Apr-2026	1 Week	None
9	Documentation: Final Report Writing & Documentation	30-Mar-2026	20-Apr-2026	3 Weeks	None

Timeline Narrative:

Weeks 1-3 (Discovery & Design): The project initiated with thorough literature research and requirements gathering, which fed directly into the system design phase (UI mockups and Database schema creation).

Weeks 4-9 (Development Sprints): Following the Agile methodology, active coding commenced with Sprint 1 focusing on the backend infrastructure. This was sequentially followed by Sprints 2 and 3 to handle the core React frontend, centralized Redux state, and PayPal integrations. Sprint 4 was dedicated to advanced feature integration, specifically an AI Chatbot utilizing the Gemini API and vector embeddings.

Weeks 10-12 (Testing & Delivery): The final stages were reserved for comprehensive System Testing and User Acceptance Testing (UAT). A dedicated contingency buffer was planned for the final week to address unexpected deployment issues, while the final report writing was strategically overlapped with the latter stages of development to ensure timely submission on April 20, 2026.

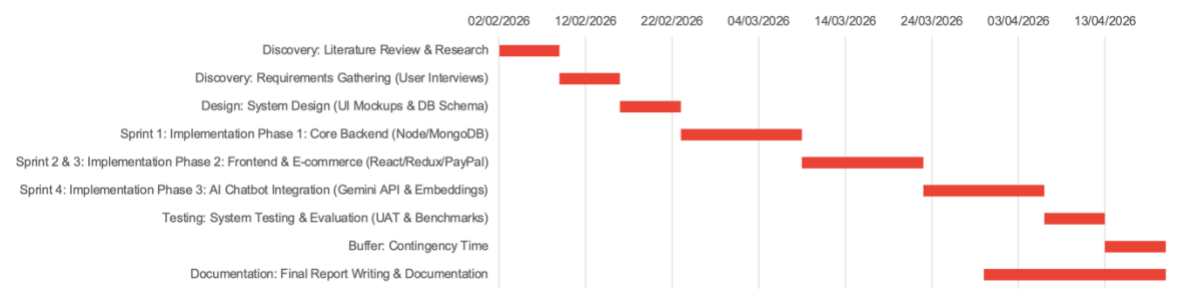


FIGURE 5

4.3 Feasibility Analysis

Before commencing full-scale development, a feasibility analysis was conducted to ensure that the TDT-Stadium project was viable within the constraints of the final-year timeframe and available resources. The assessment evaluated three primary dimensions: Technical, Economic, and Operational feasibility.

Technical Feasibility

The project was deemed highly technically feasible. The chosen technology stack—comprising MongoDB, Express.js, React.js, and Node.js (MERN), along with Vite and Redux—relies on well-documented, industry-standard frameworks. The skills required to implement these technologies align directly with the learning outcomes of the BSc Hons Computing program. Furthermore, all necessary development tools, including Visual Studio Code, Git/GitHub, and Postman for API testing, are readily available and fully supported on standard development machines. The integration of third-party services, such as the PayPal Developer API and Gemini AI API, is facilitated by comprehensive official documentation and robust community support, minimizing technical risk.

Economic Feasibility

From an economic standpoint, TDT-Stadium is highly viable as it operates well within a zero-to-minimal budget constraint typical of a student project. The project leverages "freemium" models and open-source software for all critical infrastructure:

- **Database:** MongoDB Atlas provides a generous free tier (M0 Sandbox) sufficient for development and testing data.
- **Hosting/Deployment:** Platforms such as Vercel (for the React frontend) and Render or Heroku (for the Node.js backend) offer free tiers suitable for prototyping and final presentation.
- **APIs:** The PayPal Developer environment operates entirely in sandbox mode, incurring no real-world financial transaction fees during testing.
- Therefore, the economic barrier to completing and deploying this prototype is essentially non-existent.

Operational Feasibility

The operational feasibility evaluates whether the system will be usable, manageable, and maintainable once deployed. TDT-Stadium is designed with a strong focus on User Experience (UX), utilizing a responsive, mobile-first design that ensures accessibility for the target audience of athletes and parents. For the store administrators, the inclusion of a dedicated, GUI-based admin dashboard (developed in Sprint 4) means that inventory management, order tracking, and user administration can be performed efficiently without requiring backend code manipulation. The modular nature of the React components and centralized Redux state also ensures that future updates and maintenance can be conducted smoothly.

4.4 Evaluation Plan & Success Metrics

To objectively determine whether the TDT-Stadium project achieved its stated aim and specific objectives (outlined in Section 1.4), a comprehensive evaluation plan was established before the development phase. The project's success is measured against a set of predefined quantitative (data-driven) and qualitative (user-driven) metrics.

Quantitative Success Metrics

These metrics evaluate the technical performance, reliability, and objective usability of the platform:

- **Performance Benchmarks:** The React frontend (bundled via Vite) must achieve an initial "Time to Interactive" (TTI) of under 2.5 seconds on standard broadband. Backend API endpoints (Node.js/Express) must return data queries in under 500 milliseconds.
- **Task Completion Rate (UAT):** During User Acceptance Testing (UAT), test users must achieve at least an 85% success rate when attempting critical path tasks without assistance, such as "filtering boots by ground type" and "completing a checkout process via PayPal."
- **Security & Testing:** The platform must pass 100% of critical security test cases, specifically proving that unauthorized users cannot access protected administrative dashboard routes or manipulate other users' cart data.

Qualitative Success Metrics

These metrics assess user satisfaction, perceived value, and the effectiveness of the user interface (UI) in solving the problems identified during the market analysis:

- **User Satisfaction Rating:** Receive an average System Usability Scale (SUS) or post-test survey score of at least 4 out of 5 from target persona representatives (e.g., amateur athletes, parents).
- **Feature Validation:** Gather explicit positive feedback regarding the platform's "Technical Specifications" presentation (e.g., glove grip ratings), proving that the platform successfully addressed the "information gap" found in generic competitor websites.
- **Administrative Usability:** Receive confirmation from testing that the custom Admin Dashboard is intuitive, allowing for the management of products and orders without requiring backend technical knowledge.

5. System Design and Architecture

5.1 Rich Picture

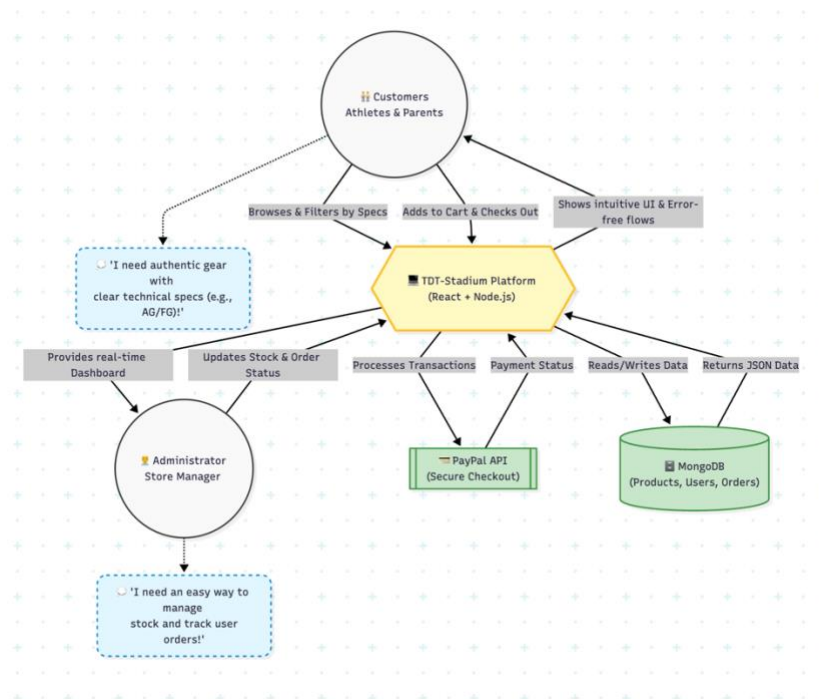


FIGURE 6

The Rich Picture illustrates the holistic environment of the TDT-Stadium project. It highlights the two primary user groups: the Customers (comprising athletes and parents) and the Administrator (store manager). The diagram captures the core concerns driving the system's development: customers are frustrated by generic shops and need specialized technical specifications to make informed purchases, while the administrator requires a streamlined dashboard to manage complex inventory without technical hassle. The central node is the TDT-Stadium Platform, acting as the bridge that resolves these concerns. It interfaces with external infrastructure, notably the MongoDB Database for flexible data storage and the PayPal API to guarantee secure financial transactions, ensuring a seamless flow of data and trust between all parties.

5.2 System Architecture

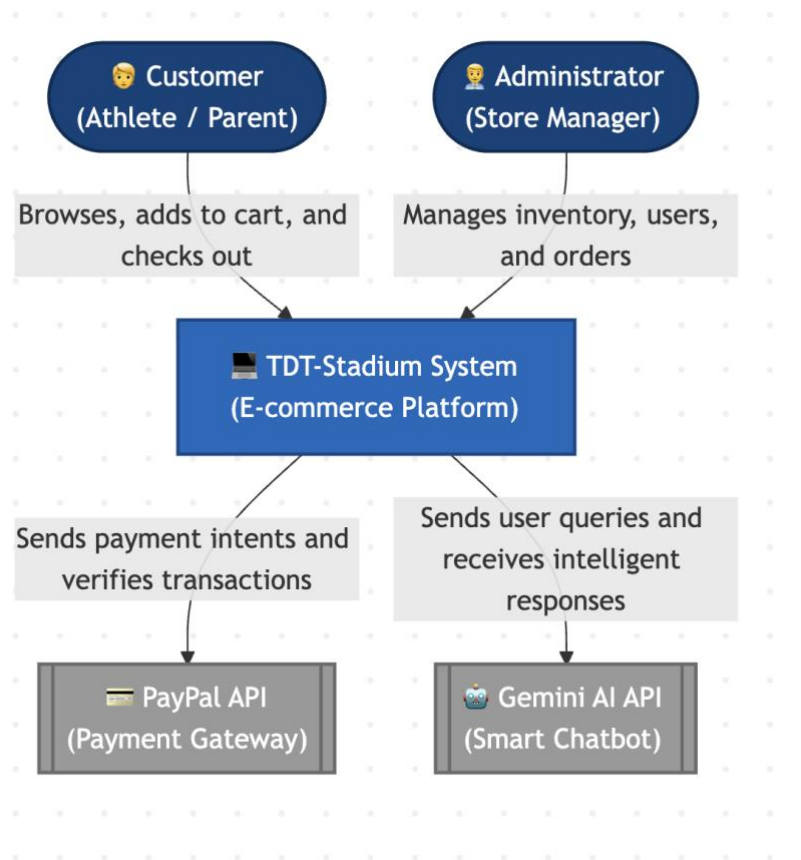


FIGURE 7

The System Context Diagram shows the two primary actors interacting with the central TDT-Stadium platform: the Customer and the Administrator. The system abstracts all internal complexity from these users. To fulfil its business requirements, TDT-Stadium relies on two external software systems: the PayPal API for processing secure financial transactions during checkout, and the Gemini AI API (integrated during Sprint 4) to power the intelligent customer support chatbot.

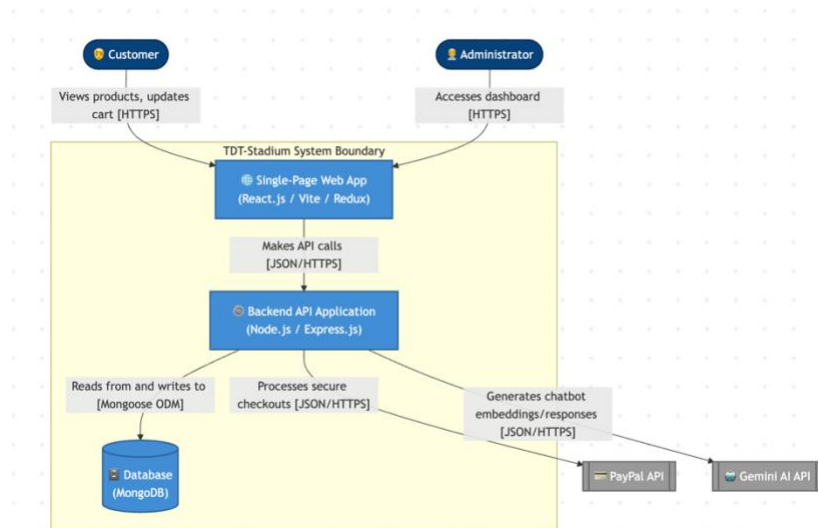


FIGURE 8

The Container Diagram reveals the three fundamental pillars of the MERN stack implementation:

Single-Page Web Application (Frontend): Built with React.js and Vite, this container runs entirely in the user's browser. It handles the UI, form validation, and local state management via Redux Toolkit.

Backend API Application (Server): Built with Node.js and Express.js, this container acts as the secure middleman. It intercepts requests from the frontend, enforces JWT authentication (via `authMiddleware.js`), executes business logic (like calculating cart totals), and communicates with external APIs.

Database: A MongoDB NoSQL database that stores all persistent data, including user profiles, hashed passwords, dynamic product catalogs, and order histories.

5.3 Technology Stack

The selection of the technology stack for TDT-Stadium was driven by the requirements for high performance, scalability, and the ability to handle complex data structures inherent to an e-commerce platform. The "MERN" stack (MongoDB, Express.js, React, Node.js) was chosen as the foundation.

The table below details the specific technologies utilized, their version contexts (as defined in the project's dependency files), the justification for their selection, and their corresponding academic or technical references.

Category	Technology	Justification	References
Frontend UI	React.js (v18) & Vite	React enables a component-based architecture for reusable UI elements (like product cards). Vite was chosen over Create React App (CRA) for significantly faster bundling and Hot Module Replacement (HMR) during development.	Meta Platforms, Inc. (2026); Vite (2026)
State Management	Redux Toolkit (@reduxjs/toolkit)	Essential for managing global application state, ensuring that the shopping cart and user authentication status remain synchronized across all pages without excessive prop-drilling.	Redux (2026)
Backend Environment	Node.js & Express.js	Provides a unified "JavaScript Everywhere" approach. Express.js offers a lightweight, fast framework for routing and building the RESTful APIs required for the storefront and admin dashboard.	OpenJS Foundation (2026)
Database	MongoDB & Mongoose	A NoSQL document database is ideal for agile development and storing diverse product attributes (e.g., boot ground types vs. glove grip). Mongoose provides straightforward schema validation and object data modeling.	MongoDB, Inc. (2026)
Authentication & Security	JSON Web Token (JWT) & bcryptjs	bcryptjs is used to cryptographically hash passwords before DB storage. JWT is used to issue secure, stateless tokens to verify user	IETF (2015)

		identity for protected API routes.	
Payment Gateway	PayPal Developer API (@paypal/react-paypal-js)	Provides a trusted, secure, and internationally recognized payment processing system, offloading the PCI compliance and financial security risks from the local server.	PayPal Developer (2026)
AI Integration	Google Gemini API	Integrated during Sprint 4 to power the intelligent customer support chatbot and generate text embeddings for advanced product search capabilities.	Google for Developers (2026)
Version Control & Management	Git & GitHub	The industry standard for source code management, tracking incremental changes, and facilitating agile task management via GitHub Projects.	Chacon and Straub (2014)

5.4 Detailed Design Diagrams

Entity Relationship Diagram (ERD): While MongoDB is a NoSQL database and does not use strict relational tables, defining the data structure and document references is critical for maintaining data integrity. The ERD below illustrates the schema design utilizing Mongoose ODM, showing the primary collections (User, Product, Cart, Order) and how they reference each other via ObjectIDs.

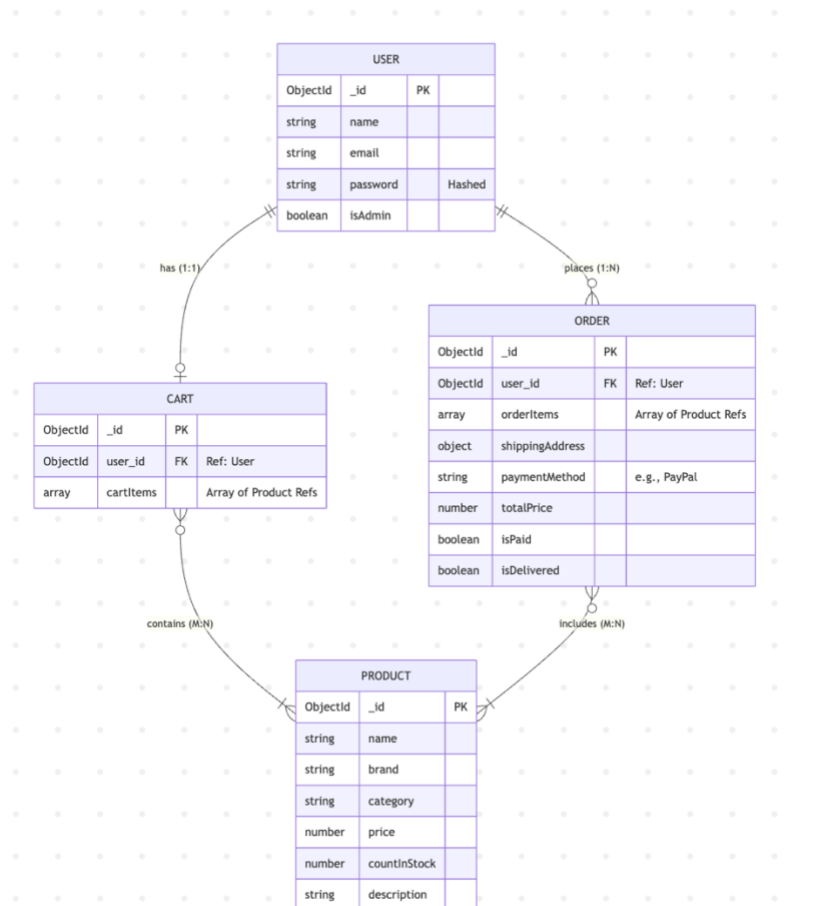


FIGURE 9

The core of the database revolves around the User collection. A user can have one active Cart at a time (1:1 relationship) and can place multiple Orders over their lifetime (1:N relationship). Both the Cart and Order documents hold arrays of references to the Product collection, enabling a Many-to-Many (M:N) relationship where multiple orders can contain multiple identical products. The isAdmin Boolean flag in the User model acts as the primary access control metric for the administrative dashboard.

Use Case Diagram: The Use Case Diagram models the system's dynamic behavior, defining exactly what actions each type of user (Actor) can perform within the system boundaries.

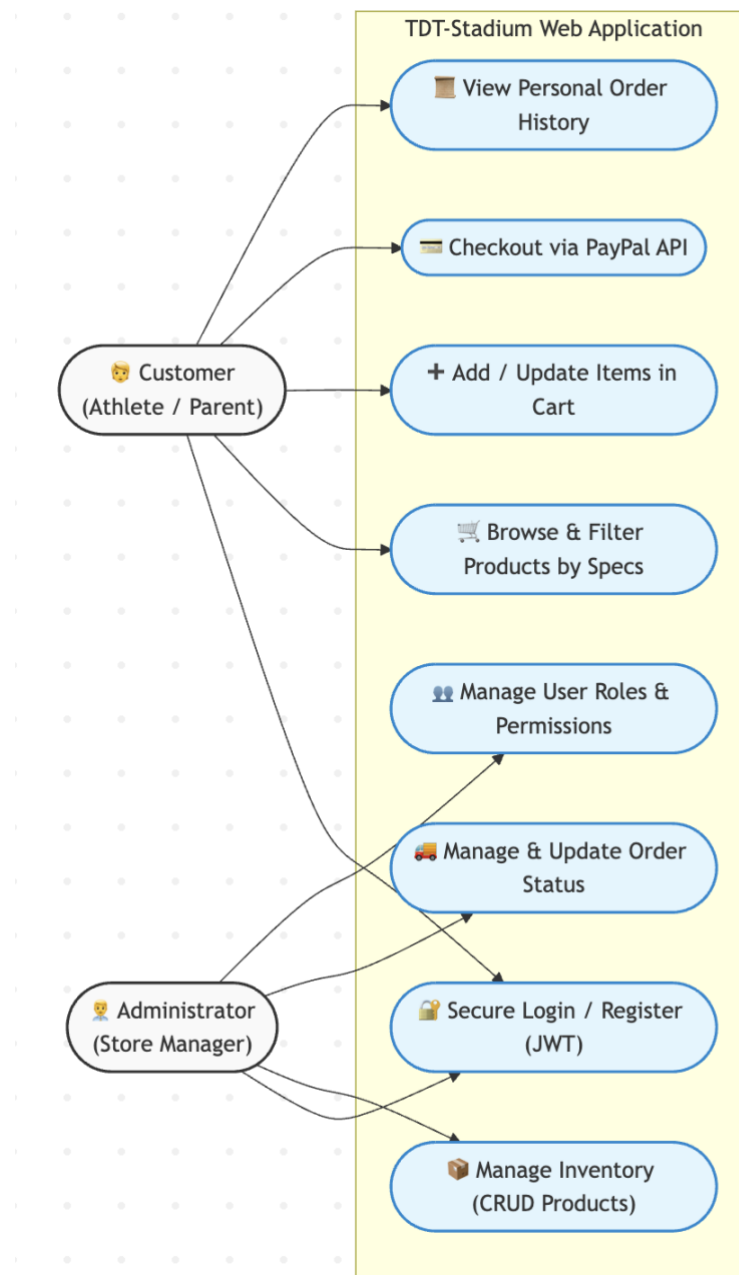


FIGURE 10

The diagram maps out the required functionalities for both the Customer and the Administrator. Shared Actions: Both actors must authenticate via the secure JWT login module (UC_Auth). Customer Actions: Customers are restricted to frontend interactions: browsing specialized products, managing their shopping cart, executing secure payments via the PayPal integration, and reviewing their historical orders.

Administrator Actions: Administrators bypass the standard storefront and access a protected dashboard. Their use cases revolve around backend management: creating and updating product inventory, processing order fulfilment statuses, and managing general user accounts.

5.5 API Design

Id	URL	Method	Description	Params (Body/Query)	Returns (JSON)
1	/api/users/register	POST	Register a new user	Body: username, email, password	{ user: {...}, token }
2	/api/users/login	POST	Authenticate user and get token	Body: email, password	{ user: {...}, token }
3	/api/users/profile	GET	Get user profile	None (Header: Authorization)	{ user object }
4	/api/users/profile	PUT	Update user profile	Body: username, email, password (optional)	{ _id, username, email, role }
5	/api/products/most-popular	GET	Get most popular products (3-5 per category)	None	[{ product objects with totalSold, hasDeal }]
6	/api/products/featured/deals	GET	Get products with "Deal" tag	None	[{ product objects }]
7	/api/products/featured/new	GET	Get products with "New" tag	None	[{ product objects }]
8	/api/products/featured/bestsellers	GET	Get products with "Best Seller" tag	None	[{ product objects }]
9	/api/products	GET	Get all products with filtering, sorting, and pagination	Query: category, brand, age, tags, minPrice, maxPrice, outsole, material, player, size, gloveSize, color, accessoryType, searchQuery, sortBy, order, page, limit	{ products[], currentPage, totalPages, totalProducts }
10	/api/products/:id	GET	Get single product by ID	Param: id	{ product object }

11	/api/products/:id/related	GET	Get related products	Param: id	[{ related product objects }]
12	/api/products	POST	Create a new product (Admin)	Body: name, description, originalPrice, discount, sku, category...	{ new product object }
13	/api/products/:id	PUT	Update a product (Admin)	Param: id Body: name, description...	{ updated product object }
14	/api/products/:id	DELETE	Delete a product (Admin)	Param: id	{ message }
15	/api/products/:id/stock	PATCH	Update stock for a specific color/size combination (Admin)	Param: id Body: colorIndex, sizeIndex, stock, accessorySize	{ updated product object }
16	/api/orders	GET	Get orders for authenticated user	None (Header: Authorization)	[{ order objects }]
17	/api/orders/:id	GET	Get order details by ID	Param: id	{ order object }
18	/api/admin/users	GET	Get all users (Admin only)	None	[{ user objects without password }]
19	/api/admin/users	POST	Create a new user (Admin only)	Body: username, email, password, role	{ _id, username, email, role, createdAt }
20	/api/admin/users/:id	PUT	Update a user (Admin only)	Param: id Body: username, email, role, password	{ _id, username, email, role, createdAt }
21	/api/admin/users/:id	DELETE	Delete a user (Admin only)	Param: id	{ message }

22	/api/admin/orders	GET	Get all orders (Admin only)	None	[{ order objects with populated user }]
23	/api/admin/orders/:id	PUT	Mark order as delivered (Admin only)	Param: id Body: status	{ updated order object }
24	/api/admin/orders/:id	DELETE	Delete an order (Admin only)	Param: id	{ message }
25	/api/admin/products	GET	Get all products (Admin only)	None	[{ product objects }]
26	/api/admin/products/:id	GET	Get a single product by ID (Admin only)	Param: id	{ product object }
27	/api/admin/products	POST	Create a new product (Admin only)	Body: product fields	{ message, product }
28	/api/admin/products/:id	PUT	Update a product (Admin only)	Param: id Body: updated fields	{ message, product }
29	/api/admin/products/:id	DELETE	Delete a product (Admin only)	Param: id	{ message }
30	/api/addresses	GET	Get all addresses for authenticated user	None	[{ address objects }]
31	/api/addresses	POST	Create a new address	Body: label, firstName, lastName, address, city, postalCode, country, phone, isDefault	{ newly created address object }
32	/api/addresses/:id	PUT	Update an address	Param: id	{ updated address object }

				Body: address fields	
33	/api/addresses/:id	DELETE	Delete an address	Param: id	{ message }
34	/api/addresses/:id/set-default	PUT	Set an address as default	Param: id	{ updated address object }
35	/api/cart	POST	Add product to cart for guest or authenticated user	Body: productId, quantity, size, color, guestId, userId	{ cart object }
36	/api/cart	GET	Get user's or guest's cart	Query: userId, guestId	{ cart object }
37	/api/cart	PUT	Update product quantity in cart	Body: productId, quantity, size, color, guestId, userId	{ cart object }
38	/api/cart	DELETE	Remove product from cart	Body: productId, size, color, guestId, userId	{ cart object }
39	/api/cart/merge	POST	Merge guest cart into user cart upon login	Body: guestId	{ merged cart object }
40	/api/checkout	POST	Create a new checkout session and order	Body: checkoutItems, shippingAddress, paymentMethod, totalPrice	{ checkout object }
41	/api/checkout/:id	GET	Get checkout by ID	Param: id	{ checkout object }
42	/api/checkout/:id/pay	PUT	Update checkout as paid and create order	Param: id Body: paymentStatus, paymentDetails	{ checkout, message }
43	/api/checkout/:id/finalize	POST	Finalize the order after successful payment	Param: id	{ finalOrder object }
44	/api/paypal/create-order	POST	Create PayPal order	Body: amount	{ orderID }

45	/api/paypal/capture-order/:orderID	POST	Capture PayPal order	Param: orderID	{ capture response object }
46	/api/chat	POST	Send a message to the AI Chatbot	Body: message	{ reply, products[] }
47	/api/contact	POST	Send contact form email	Body: name, email, phone, message	{ success, message }
48	/api/subscribers	POST	Create a new subscriber	Body: email	{ message, subscriber object }
49	/api/upload	POST	Upload an image to Cloudinary	Body: form-data (image)	{ imageUrl }

6. Implementation

6.1 Development Environment

To ensure a streamlined, efficient, and error-free coding process, a robust local development environment was established before initiating the implementation phase. The environment was standardized around modern web development tools optimized for the MERN stack.

Hardware Specifications

Development was conducted on a standard personal computer running a modern operating system (e.g., Windows 11 / macOS), equipped with sufficient RAM (minimum 8GB) to concurrently run the Node.js server, the Vite React development server, and local database instances without performance bottlenecks.

Software and Tooling

The following software tools formed the backbone of the development workflow:

Integrated Development Environment (IDE): Visual Studio Code (VS Code)

VS Code was selected as the primary code editor due to its extensive ecosystem and native support for JavaScript and Node.js. Critical extensions were utilized to maintain code quality, including ESLint (for identifying JavaScript syntax errors and enforcing coding standards) and Prettier (for automatic, consistent code formatting across both frontend and backend directories).

Version Control System: Git and GitHub

Git was utilized for local source code versioning, allowing for safe experimentation and feature branching (e.g., developing the PayPal integration on a separate branch before merging into the main codebase). The repository was hosted remotely on GitHub, which served as both a secure backup and a project management tool (tracking commits and task progress in alignment with Agile Sprints).

API Development and Testing: Postman

Before connecting the React frontend to the Node.js backend, Postman was heavily utilized to test all RESTful API endpoints independently. This tool allowed for the simulation of HTTP requests (GET, POST, PUT, DELETE), the injection of JSON Web Tokens (JWT) into authorization headers, and the verification of JSON responses (as documented in the project's `API_TESTING_GUIDE.md`).

Database Management: MongoDB Compass

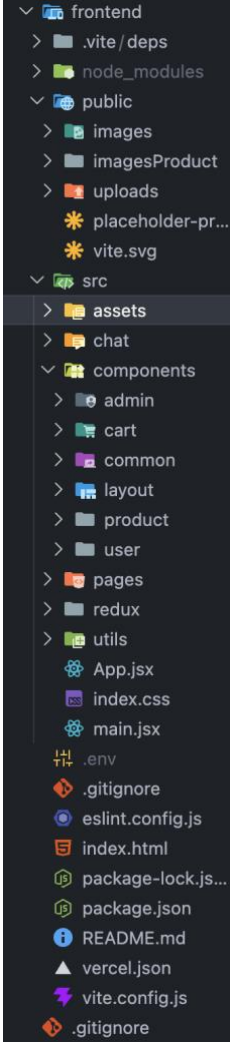
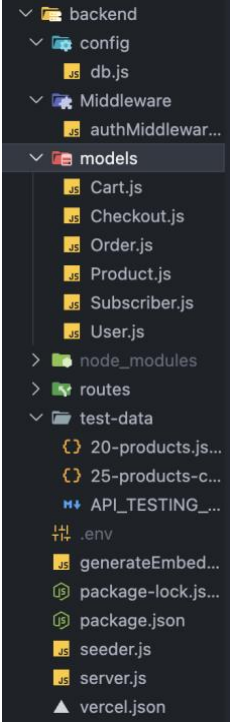
While MongoDB Atlas was used for cloud hosting, MongoDB Compass provided a graphical user interface (GUI) for local database inspection. It allowed for the visual verification of complex data structures, such as verifying that password hashes were correctly generated and that the nested arrays for product color/size variants were structured exactly as defined in the Mongoose schemas.

Package Management: npm

The Node Package Manager (npm) was used exclusively to install, manage, and update all project dependencies (e.g., express, mongoose, @reduxjs/toolkit, axios) for both the frontend and backend environments.

6.2 Backend & Frontend Implementation

The backend follows a strict modular architecture, separating routing logic, middleware, and database models. The most complex engineering challenges involved integrating Artificial Intelligence, handling dynamic data queries, and securing financial transactions.

 Front End	 Back End
---	---

AI Vector Search Integration (chatbotRoutes.js)

One of the most advanced features of the platform is the intelligent customer support chatbot. Instead of relying on rigid, pre-programmed responses, this module utilizes the Google Gemini API to generate vector embeddings of user queries and perform a sophisticated semantic search against the MongoDB database.

Complexity Highlight: The system must first parse the user's natural language, map common terms (e.g., "shoes" to "football boots"), generate a multidimensional vector, and use MongoDB's \$vectorSearch to find the mathematically closest product matches before feeding that context back to the AI for a conversational response

```
1. // backend/routes/chatbotRoutes.js
2. const express = require('express');
3. const { GoogleGenerativeAI } = require('@google/generative-ai');
4. const Product = require('../models/Product');
5. const router = express.Router();
6.
7. // Initialize Gemini SDK
8. const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
9.
10. // Helper function to detect if user is asking for products
11. const isProductQuery = (message) => {
12.   const lowerMessage = message.toLowerCase();
13.
14.   // Greetings and casual conversation (must be short and only greeting)
15.   const greetings = /\b(hi|hello|hey|good morning|good afternoon|good
evening|thanks|thank you|bye|goodbye)\b/;
16.   if (greetings.test(lowerMessage) && lowerMessage.length < 30 &&
!lowerMessage.match(/\b(boot|shoe|glove|ball|sock|play|indoor|outdoor|grass|field)\b/)) {
17.     return false;
18.   }
19.
20.   // Irrelevant requests
21.   const irrelevant = /\b(password|pass|money|cash|free|discount code|coupon|hack|credit
card|personal|address|phone|email)\b/;
22.   if (irrelevant.test(lowerMessage)) {
23.     return false;
24.   }
25.
26.   // Product-related keywords (expanded with play context)
27.   const productKeywords = /\b(show|find|looking
for|need|want|buy|purchase|recommend|suggest|boot|boots|shoe|shoes|glove|gloves|ball|balls|sock|
socks|tape|product|equipment|gear|nike|adidas|puma|under armour|mizuno|new
balance|uhlsport|thuong
dinh|play|playing|indoor|outdoor|futsal|artificial|grass|turf|field|pitch)\b/;
28.
29.   return productKeywords.test(lowerMessage);
30. };
31.
32. // Helper function to detect brand from user query
33. const detectBrand = (message) => {
34.   const lowerMessage = message.toLowerCase();
35.
36.   const brands = {
37.     'nike': 'Nike',
38.     'adidas': 'Adidas',
39.     'puma': 'Puma',
40.     'mizuno': 'Mizuno',
41.     'new balance': 'New Balance',
42.     'under armour': 'Under Armour',
43.     'uhlsport': 'Uhlsport',
44.     'thuong dinh': 'Thuong Dinh',
45.     'soka': 'Soka'
46.   };
31.
32. // Helper function to detect brand from user query
33. const detectBrand = (message) => {
34.   const lowerMessage = message.toLowerCase();
35.
36.   const brands = {
37.     'nike': 'Nike',
38.     'adidas': 'Adidas',
39.     'puma': 'Puma',
40.     'mizuno': 'Mizuno',
41.     'new balance': 'New Balance',
42.     'under armour': 'Under Armour',
43.     'uhlsport': 'Uhlsport',
44.     'thuong dinh': 'Thuong Dinh',
45.     'soka': 'Soka'
46.   };

```

```

47.
48.   for (const [key, value] of Object.entries(brands)) {
49.       if (lowerMessage.includes(key)) {
50.           return value;
51.       }
52.   }
53.
54.   return null;
55. };
56.
57. // Helper function to detect category from user query
58. const detectCategory = (message) => {
59.     const lowerMessage = message.toLowerCase();
60.
61.     // Football boots keywords
62.     if (lowerMessage.match(/\b(boot|boots|shoe|shoes|cleats|cleat|footwear|studs)\b/)) {
63.         return 'BOOTS';
64.     }
65.
66.     // Goalkeeper gloves keywords
67.     if (lowerMessage.match(/\b(glove|gloves|goalkeeper|keeper|goalie)\b/)) {
68.         return 'GLOVES';
69.     }
70.
71.     // Accessories keywords
72.     if
(lowerMessage.match(/\b(ball|balls|sock|socks|tape|tapes|accessory|accessories|equipment)\b/)) {
73.         return 'ACCESSORIES';
74.     }
75.
76.     return null; // No specific category detected
77. };
78.
79. // Helper function to enhance query for better search
80. const enhanceQuery = (message) => {
81.     let enhanced = message;
82.
83.     // Map common terms to football-specific terms
84.     const termMappings = {
85.         'shoe': 'football boot',
86.         'shoes': 'football boots',
87.         'cleat': 'football boot',
88.         'cleats': 'football boots',
89.         'keeper': 'goalkeeper',
90.         'goalie': 'goalkeeper',
91.         'in house': 'indoor',
92.         'indoor': 'indoor football boot',
93.         'futsal': 'indoor football boot',
94.         'play indoor': 'indoor football boot',
95.         'play in house': 'indoor football boot'
96.     };
97.
98.     Object.entries(termMappings).forEach(([from, to]) => {
99.         const regex = new RegExp(`\\b${from}\\b`, 'gi');
100.        enhanced = enhanced.replace(regex, to);
101.    });
102.
103.    return enhanced;
104. };
105.
106. router.post('/', async (req, res) => {
107.     try {
108.         const { message } = req.body;
109.
110.         if (!message) {
111.             return res.status(400).json({ error: "Message is required" });
112.         }
113.

```

```

114.         // Check if this is a product query
115.         const isProduct = isProductQuery(message);
116.
117.         // If not a product query, respond without products
118.         if (!isProduct) {
119.             const chatModel = genAI.getGenerativeModel({
120.                 model: "gemini-2.5-flash-lite",
121.                 systemInstruction: `You are a helpful AI assistant for TDT Stadium, a
football equipment store.
122.
123. IMPORTANT RULES:
124. 1. For greetings (hi, hello, hey), respond warmly and briefly introduce yourself
125. 2. For thank you messages, respond politely
126. 3. For irrelevant requests (passwords, money, personal info, discounts codes), politely
decline and explain you can only help with product recommendations
127. 4. Keep responses very brief (1-2 sentences)
128. 5. Always be friendly and professional
129.
130. RESPONSE EXAMPLES:
131. - "Hi" → "Hello! I'm the TDT Stadium assistant. I can help you find football boots, gloves,
and accessories. What are you looking for?"
132. - "Thanks" → "You're welcome! Let me know if you need anything else."
133. - "Give me a discount code" → "I'm sorry, I can only help you find and recommend products.
For discount codes, please contact our customer service."
134. - "What's your password?" → "I'm a product consultant and can only help you find football
equipment. How can I assist you with boots, gloves, or accessories?"
135.             });
136.
137.             const chatResult = await chatModel.generateContent(message);
138.             const botReply = chatResult.response.text();
139.
140.             return res.json({
141.                 reply: botReply,
142.                 products: [] // No products for non-product queries
143.             });
144.         }
145.
146.         // Detect category and brand from user query
147.         const detectedCategory = detectCategory(message);
148.         const detectedBrand = detectBrand(message);
149.
150.         // Enhance query for better search results
151.         const enhancedMessage = enhanceQuery(message);
152.
153.         // 1. Convert user's message into an embedding vector
154.         const model = genAI.getGenerativeModel({ model: "gemini-embedding-001" });
155.         const result = await model.embedContent(enhancedMessage);
156.         const queryVector = result.embedding.values;
157.
158.         // 2. Perform Vector Search in MongoDB
159.         let relevantProducts = await Product.aggregate([
160.             {
161.                 "$vectorSearch": {
162.                     "index": "vector_index",
163.                     "path": "embedding",
164.                     "queryVector": queryVector,
165.                     "numCandidates": 20,
166.                     "limit": 8 // Get more products initially
167.                 }
168.             },
169.             {
170.                 "$project": {
171.                     "_id": 1,
172.                     "name": 1,
173.                     "description": 1,
174.                     "price": 1,
175.                     "category": 1,
176.                     "brand": 1,

```

```

177.         "player": 1,
178.         "outsole": 1,
179.         "countInStock": 1,
180.         "images": 1,
181.         "colorVariants": 1
182.     }
183. }
184. });
185.
186. // 3. Filter products by detected category and brand
187. if (detectedCategory && relevantProducts.length > 0) {
188.     const categoryFiltered = relevantProducts.filter(p => p.category ===
detectedCategory);
189.
190.     // Only use filtered results if we found products in that category
191.     if (categoryFiltered.length > 0) {
192.         relevantProducts = categoryFiltered;
193.     }
194. }
195.
196. // Filter by brand if detected
197. if (detectedBrand && relevantProducts.length > 0) {
198.     const brandFiltered = relevantProducts.filter(p => p.brand === detectedBrand);
199.
200.     // Only use brand filtered results if we found products from that brand
201.     if (brandFiltered.length > 0) {
202.         relevantProducts = brandFiltered;
203.     }
204. }
205.
206. // Limit to maximum 4 products, but can be less
207. relevantProducts = relevantProducts.slice(0, 4);
208.
209. // 4. Format the retrieved products into a text string
210. let contextText = "";
211.
212. if (relevantProducts.length === 0) {
213.     contextText = "No matching products found in the database.\n\n";
214. } else {
215.     contextText = `Found ${relevantProducts.length} relevant product(s):\n\n`;
216.     relevantProducts.forEach((product, index) => {
217.         contextText += `${index + 1}. ${product.name}\n`;
218.         contextText += `    Category: ${product.category}\n`;
219.         contextText += `    Brand: ${product.brand}\n`;
220.         contextText += `    Price: ${product.price}\n`;
221.         contextText += `    Description: ${product.description}\n`;
222.         if (product.player && product.player !== "None") {
223.             contextText += `    Associated Player: ${product.player}\n`;
224.         }
225.         if (product.outsole) {
226.             contextText += `    Outsole: ${product.outsole}\n`;
227.         }
228.         if (product.colorVariants && product.colorVariants.length > 0) {
229.             const colors = product.colorVariants.map(v => v.color).join(', ');
230.             contextText += `    Available Colors: ${colors}\n`;
231.         }
232.         contextText += `    In Stock: ${product.countInStock > 0 ? 'Yes' : 'No'}\n`;
233.         contextText += '\n';
234.     });
235. }
236.
237. // Add context about what user is looking for
238. let userContext = `User is looking for: ${message}\n`;
239. if (detectedCategory) {
240.     userContext += `Detected Category: ${detectedCategory}\n`;
241. }
242. if (detectedBrand) {
243.     userContext += `Detected Brand: ${detectedBrand}\n`;

```

```

244.     }
245.     userContext += `\n`;
246.
247.     // 5. Send the context and user query to Gemini 2.5 Flash Lite
248.     const chatModel = genAI.getGenerativeModel({
249.         model: "gemini-2.5-flash-lite",
250.         systemInstruction: `You are a helpful, enthusiastic, and expert AI sales
assistant for TDT Stadium, a football equipment store.
251.
252. IMPORTANT RULES:
253. 1. Use ONLY the provided product context to recommend items
254. 2. If the user asks for "shoes" or "boots", recommend products from the BOOTS category ONLY
255. 3. If the user asks for "gloves", recommend products from the GLOVES category ONLY
256. 4. If the user asks for "balls", "socks", or "tape", recommend products from the
ACCESSORIES category ONLY
257. 5. If user specifies a brand (Nike, Adidas, Puma, etc.), ONLY recommend products from that
brand
258. 6. If user mentions "indoor", "in house", "futsal", recommend boots with Indoor outsole
259. 7. If user mentions "artificial grass", "turf", "AG", recommend boots with Artificial Grass
outsole
260. 8. If user mentions "firm ground", "grass", "FG", recommend boots with Firm Ground outsole
261. 9. If the provided products don't match the user's request (wrong category, brand, or
surface), politely explain that we don't have exactly what they're looking for
262. 10. Always mention the product category, brand, and outsole type when recommending boots
263. 11. Keep responses friendly, concise, and easy to read (2-3 sentences max)
264. 12. Do not make up product details or prices
265. 13. If showing multiple products, briefly highlight what makes each one special
266. 14. The number of products can vary (1-4), don't always expect 4 products
267.
268. RESPONSE FORMAT:
269. - Start with a friendly acknowledgment of their request
270. - Recommend the products that match their needs
271. - Mention key features (brand, price, outsole type, special attributes)
272. - Keep it conversational and helpful
273. - If only 1-2 products found, that's fine - recommend what's available`
274.     });
275.
276.     const prompt = `${userContext}${contextText}\n\nProvide a helpful recommendation
based on these products. Remember to only recommend products that match the user's category and
brand requirements.`;
277.
278.     const chatResult = await chatModel.generateContent(prompt);
279.     const botReply = chatResult.response.text();
280.
281.     // Send the AI's response back to the frontend
282.     res.json({
283.         reply: botReply,
284.         products: relevantProducts
285.     });
286.
287. } catch (error) {
288.     console.error("Chatbot Error:", error);
289.     res.status(500).json({ error: "Something went wrong with the AI assistant." });
290. }
291. });
292.
293. module.exports = router;
294.
295.

```

Secure Server-to-Server Payment Processing (paypalRoutes.js)

To ensure PCI compliance and financial security, checkout operations are strictly handled server-side rather than relying entirely on the frontend.

Complexity Highlight: This component must authenticate with the PayPal OAuth2 server using securely stored environment variables (PAYPAL_CLIENT_ID and SECRET) to generate a temporary access_token. It then constructs a strict payload to create an order intent. If the token expires or the API fails, the backend must gracefully handle the error without exposing sensitive API keys to the client.

```
1. const express = require("express");
2. const axios = require("axios");
3. const { protect } = require("../Middleware/authMiddleware");
4.
5. const router = express.Router();
6.
7. // PayPal API base URL
8. const PAYPAL_API = process.env.PAYPAL_MODE === "live"
9.   ? "https://api-m.paypal.com"
10.   : "https://api-m.sandbox.paypal.com";
11.
12. // Get PayPal access token
13. const getPayPalAccessToken = async () => {
14.   const auth = Buffer.from(
15.     `${process.env.PAYPAL_CLIENT_ID}:${process.env.PAYPAL_CLIENT_SECRET}`
16.   ).toString("base64");
17.
18.   try {
19.     const response = await axios.post(
20.       `${PAYPAL_API}/v1/oauth2/token`,
21.       "grant_type=client_credentials",
22.       {
23.         headers: {
24.           Authorization: `Basic ${auth}`,
25.           "Content-Type": "application/x-www-form-urlencoded",
26.         },
27.       }
28.     );
29.     return response.data.access_token;
30.   } catch (error) {
31.     console.error("Error getting PayPal access token:", error.response?.data ||
error.message);
32.     throw new Error("Failed to authenticate with PayPal");
33.   }
34. };
35.
36. // @route POST /api/paypal/create-order
37. // @desc Create PayPal order
38. // @access Private
39. router.post("/create-order", protect, async (req, res) => {
40.   const { amount } = req.body;
41.
42.   if (!amount || amount <= 0) {
43.     return res.status(400).json({ message: "Invalid amount" });
44.   }
45.
46.   try {
47.     const accessToken = await getPayPalAccessToken();
48.
49.     const response = await axios.post(
50.       `${PAYPAL_API}/v2/checkout/orders`,
51.       {
52.         intent: "CAPTURE",
53.         purchase_units: [
54.
```

```

55.             amount: {
56.                 currency_code: "USD",
57.                 value: amount.toFixed(2),
58.             },
59.         },
60.     ],
61. },
62. {
63.     headers: {
64.         Authorization: `Bearer ${accessToken}`,
65.         "Content-Type": "application/json",
66.     },
67. }
68. );
69.
70.     res.json({ orderID: response.data.id });
71. } catch (error) {
72.     console.error("Error creating PayPal order:", error.response?.data ||
error.message);
73.     res.status(500).json({
74.         message: "Failed to create PayPal order",
75.         error: error.response?.data || error.message
76.     });
77. }
78. });
79.
80. // @route POST /api/paypal/capture-order/:orderID
81. // @desc Capture PayPal order
82. // @access Private
83. router.post("/capture-order/:orderID", protect, async (req, res) => {
84.     const { orderID } = req.params;
85.
86.     if (!orderID) {
87.         return res.status(400).json({ message: "Order ID is required" });
88.     }
89.
90.     try {
91.         const accessToken = await getPayPalAccessToken();
92.
93.         const response = await axios.post(
94.             `${PAYPAL_API}/v2/checkout/orders/${orderID}/capture`,
95.             {},
96.             {
97.                 headers: {
98.                     Authorization: `Bearer ${accessToken}`,
99.                     "Content-Type": "application/json",
100.                 },
101.             }
102.         );
103.
104.         res.json(response.data);
105.     } catch (error) {
106.         console.error("Error capturing PayPal order:", error.response?.data ||
error.message);
107.         res.status(500).json({
108.             message: "Failed to capture PayPal order",
109.             error: error.response?.data || error.message
110.         });
111.     }
112. });
113.
114. module.exports = router;
115.
116.

```

Centralized Cart State Management (cartSlice.js)

In an e-commerce application, the shopping cart state must be instantly accessible from the navigation bar, the checkout page, and individual product pages.

Complexity Highlight: Using Redux Toolkit (@reduxjs/toolkit), the cartSlice manages not only the local arrays of items but also handles asynchronous thunks (createAsyncThunk) that sync the local state with the MongoDB database. The logic must calculate total prices dynamically and handle the complex edge case of merging a "Guest Cart" with an "Authenticated User Cart" upon login without losing data.

```
1. import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
2. import axios from "axios";
3.
4. // Constants
5. const API_URL = `${import.meta.env.VITE_BACKEND_URL}/api/cart`;
6.
7. // Helper function to get token
8. const getAuthToken = () => JSON.parse(localStorage.getItem("userToken"));
9.
10. // Helper function to get cart from localStorage
11. const loadCartFromStorage = () => {
12.   const cartData = localStorage.getItem("cart");
13.   return cartData ? JSON.parse(cartData) : { products: [], totalPrice: 0 };
14. };
15.
16. // Helper function to save cart to localStorage
17. const saveCartToStorage = (cart) => {
18.   localStorage.setItem("cart", JSON.stringify(cart));
19. };
20.
21. // Fetch cart for user or guest
22. export const fetchCart = createAsyncThunk(
23.   "cart/fetchCart",
24.   async ({ userId, guestId }, { rejectWithValue }) => {
25.     try {
26.       const response = await axios.get(API_URL, {
27.         params: { userId, guestId },
28.       });
29.       return response.data;
30.     } catch (error) {
31.       return rejectWithValue(
32.         error.response && error.response.data.message
33.           ? error.response.data.message
34.           : error.message
35.       );
36.     }
37.   }
38. );
39.
40. // Add an item to the cart
41. export const addToCart = createAsyncThunk(
42.   "cart/addToCart",
43.   async ({ productId, quantity, size, color, guestId, userId }, { rejectWithValue }) => {
44.     try {
45.       const response = await axios.post(API_URL, {
46.         productId,
47.         quantity,
48.         size,
49.         color,
50.         guestId,
51.         userId
52.       });
53.       return response.data;
54.     } catch (error) {
55.       return rejectWithValue(
```

```

56.         error.response && error.response.data.message
57.         ? error.response.data.message
58.         : error.message
59.     );
60. }
61. }
62. );
63.
64. // Update the quantity of an item in the cart
65. export const updateCartItem = createAsyncThunk(
66.     "cart/updateCartItem",
67.     async ({ productId, quantity, size, color, guestId, userId }, { rejectWithValue }) => {
68.         try {
69.             const response = await axios.put(API_URL, {
70.                 productId,
71.                 quantity,
72.                 size,
73.                 color,
74.                 guestId,
75.                 userId
76.             });
77.             return response.data;
78.         } catch (error) {
79.             return rejectWithValue(
80.                 error.response && error.response.data.message
81.                 ? error.response.data.message
82.                 : error.message
83.             );
84.         }
85.     }
86. );
87.
88. // Remove an item from the cart
89. export const removeFromCart = createAsyncThunk(
90.     "cart/removeFromCart",
91.     async ({ productId, size, color, guestId, userId }, { rejectWithValue }) => {
92.         try {
93.             const response = await axios.delete(API_URL, {
94.                 data: { productId, size, color, guestId, userId },
95.             });
96.             return response.data;
97.         } catch (error) {
98.             return rejectWithValue(
99.                 error.response && error.response.data.message
100.                 ? error.response.data.message
101.                 : error.message
102.             );
103.         }
104.     }
105. );
106.
107. // Merge guest cart with user cart on login
108. export const mergeCart = createAsyncThunk(
109.     "cart/mergeCart",
110.     async ({ guestId, userId }, { rejectWithValue }) => {
111.         try {
112.             const response = await axios.post(
113.                 `${API_URL}/merge`,
114.                 { guestId, userId },
115.                 {
116.                     headers: {
117.                         Authorization: `Bearer ${getAuthToken()}`
118.                     }
119.                 }
120.             );
121.             return response.data;
122.         } catch (error) {
123.             return rejectWithValue(

```

```

124.         error.response ?? error.response.data.message
125.         ? error.response.data.message
126.         : error.message
127.     );
128.   }
129. }
130. );
131.
132. // Slice
133. const cartSlice = createSlice({
134.   name: "cart",
135.   initialState: {
136.     cart: loadCartFromStorage(),
137.     loading: false,
138.     error: null,
139.   },
140.   reducers: {
141.     clearCart: (state) => {
142.       state.cart = { products: [] };
143.       localStorage.removeItem("cart");
144.     },
145.   },
146.   extraReducers: (builder) => {
147.     builder
148.       .addCase(fetchCart.pending, (state) => {
149.         state.loading = true;
150.         state.error = null;
151.       })
152.       .addCase(fetchCart.fulfilled, (state, action) => {
153.         state.loading = false;
154.         state.cart = action.payload;
155.         saveCartToStorage(action.payload);
156.       })
157.       .addCase(fetchCart.rejected, (state, action) => {
158.         state.loading = false;
159.         state.error = action.payload || "Failed to fetch cart";
160.       })
161.       .addCase(addToCart.pending, (state) => {
162.         state.loading = true;
163.         state.error = null;
164.       })
165.       .addCase(addToCart.fulfilled, (state, action) => {
166.         state.loading = false;
167.         state.cart = action.payload;
168.         saveCartToStorage(action.payload);
169.       })
170.       .addCase(addToCart.rejected, (state, action) => {
171.         state.loading = false;
172.         state.error = action.payload || "Failed to add to cart";
173.       })
174.       .addCase(updateCartItem.pending, (state) => {
175.         state.loading = true;
176.         state.error = null;
177.       })
178.       .addCase(updateCartItem.fulfilled, (state, action) => {
179.         state.loading = false;
180.         state.cart = action.payload;
181.         saveCartToStorage(action.payload);
182.       })
183.       .addCase(updateCartItem.rejected, (state, action) => {
184.         state.loading = false;
185.         state.error = action.payload || "Failed to update cart item";
186.       })
187.       .addCase(removeFromCart.pending, (state) => {
188.         state.loading = true;
189.         state.error = null;
190.       })
191.       .addCase(removeFromCart.fulfilled, (state, action) => {

```

```

192.         state.loading = false;
193.         state.cart = action.payload;
194.         saveCartToStorage(action.payload);
195.     })
196.     .addCase(removeFromCart.rejected, (state, action) => {
197.         state.loading = false;
198.         state.error = action.payload || "Failed to remove from cart";
199.     })
200.     .addCase(mergeCart.pending, (state) => {
201.         state.loading = true;
202.         state.error = null;
203.     })
204.     .addCase(mergeCart.fulfilled, (state, action) => {
205.         state.loading = false;
206.         state.cart = action.payload;
207.         saveCartToStorage(action.payload);
208.     })
209.     .addCase(mergeCart.rejected, (state, action) => {
210.         state.loading = false;
211.         state.error = action.payload || "Failed to merge cart";
212.     });
213. },
214. });
215.
216. export const { clearCart } = cartSlice.actions;
217. export default cartSlice.reducer;
218.
219.

```

Third-Party SDK Integration (PaypalButton.jsx)

Integrating the @paypal/react-paypal-js SDK required carefully managing the React component lifecycle to ensure the payment buttons rendered correctly and securely.

Complexity Highlight: The component must wait for the Redux state to confirm the cart total, request an orderID from the backend (createOrder callback), and immediately upon the user approving the transaction in the PayPal popup, trigger a backend capture request (onApprove callback). State management here is critical to prevent duplicate charges or UI freezing during network delays.

```
1. import { useState, useRef, useEffect } from 'react';
2. import { useNavigate } from 'react-router-dom';
3. import { FaAngleDoubleDown } from 'react-icons/fa';
4. import { IoChatbubbleEllipsesSharp } from 'react-icons/io5';
5.
6. import axios from 'axios';
7.
8. const ChatWidget = () => {
9.   const navigate = useNavigate();
10.  const messagesEndRef = useRef(null);
11.  const messagesContainerRef = useRef(null);
12.  const [isOpen, setIsOpen] = useState(false);
13.  const [messages, setMessages] = useState([
14.    { role: 'bot', text: 'Hi there! 🤖 I am the TDT Stadium AI assistant. Looking for
specific boots, gloves, or accessories?' }
15.  ]);
16.  const [input, setInput] = useState('');
17.  const [isLoading, setIsLoading] = useState(false);
18.  const [showScrollButton, setShowScrollButton] = useState(false);
19.
20.  const toggleChat = () => setIsOpen(!isOpen);
21.
22.  // Scroll to bottom function
23.  const scrollToBottom = () => {
24.    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
25.  };
26.
27.  // Auto-scroll to bottom when new messages arrive
28.  useEffect(() => {
29.    if (messages.length > 0) {
30.      scrollToBottom();
31.    }
32.  }, [messages]);
33.
34.  // Handle scroll to show/hide scroll button
35.  const handleScroll = () => {
36.    if (messagesContainerRef.current) {
37.      const { scrollTop, scrollHeight, clientHeight } = messagesContainerRef.current;
38.      const isNearBottom = scrollHeight - scrollTop - clientHeight < 100;
39.      setShowScrollButton(!isNearBottom);
40.    }
41.  };
42.
43.  // Start new chat
44.  const handleNewChat = () => {
45.    setMessages([
46.      { role: 'bot', text: 'Hi there! 🤖 I am the TDT Stadium AI assistant. Looking for
specific boots, gloves, or accessories?' }
47.    ]);
48.    setInput('');
49.  };
50.
51.  const handleViewProduct = (productId) => {
52.    navigate(`/product/${productId}`);
53.    setIsOpen(false); // Close chat when navigating
```

```

54.   });
55.
56.   const sendMessage = async (e) => {
57.     e.preventDefault();
58.     if (!input.trim()) return;
59.
60.     const userMessage = input.trim();
61.     setMessages(prev => [...prev, { role: 'user', text: userMessage }]);
62.     setInput('');
63.     setIsLoading(true);
64.
65.     try {
66.       const backendUrl = import.meta.env.VITE_BACKEND_URL || 'http://localhost:9000';
67.       const response = await axios.post(`${backendUrl}/api/chat`, {
68.         message: userMessage
69.       });
70.
71.       setMessages(prev => [...prev, {
72.         role: 'bot',
73.         text: response.data.reply,
74.         products: response.data.products || []
75.       }]);
76.     } catch (error) {
77.       console.error("Chat error:", error);
78.       setMessages(prev => [...prev, { role: 'bot', text: 'Sorry, I am having trouble
connecting right now. Please try again later.' }]);
79.     } finally {
80.       setIsLoading(false);
81.     }
82.   });
83.
84.   return (
85.     <div className="fixed bottom-6 right-6 z-50">
86.       /* Chat Window */
87.       {isOpen && (
88.         <div className="bg-[#ffffffe] rounded-lg shadow-xl w-80 sm:w-96 mb-4 flex flex-col
relative" style={{ height: '500px' }}>
89.           /* Header */
90.           <div className="bg-[#004643] text-[#ffffffe] p-4 rounded-t-lg flex justify-between
items-center">
91.             <h3 className="font-bold">TDT Stadium Assistant</h3>
92.             <div className="flex items-center gap-2">
93.               /* New Chat Button */
94.               <button
95.                 onClick={handleNewChat}
96.                 className="text-[#ffffffe] hover:scale-110 transition-colors"
97.                 title="Start new chat"
98.               >
99.                 <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-5 h-5">
100.                   <path strokeLinecap="round" strokeLinejoin="round" d="M12 4.5v15m7.5-
7.5h-15" />
101.                 </svg>
102.               </button>
103.               /* Close Button */
104.               <button onClick={toggleChat} className="text-[#ffffffe] hover:scale-110 font-
bold">
105.                 X
106.               </button>
107.             </div>
108.           </div>
109.
110.           /* Messages Area */
111.           <div
112.             ref={messagesContainerRef}
113.             onScroll={handleScroll}
114.             className="flex-1 p-4 overflow-y-auto bg-[#ffffffe] flex flex-col gap-3
relative"

```

```

115.         >
116.         {messages.map((msg, index) => (
117.             <div key={index} className={` ${msg.role === 'user' ? 'self-end' : 'self-
118.                 start'} max-w-[85%]`} >
119.                 {/* Text Message */}
120.                 <div className={` rounded-lg p-3 ${msg.role === 'user' ? 'bg-[#004643] text-
121.                     [ffffffe]' : 'bg-[#abd1c6] text-[#001e1d]'}`} >
122.                     <p className="text-sm whitespace-pre-wrap">{msg.text}</p>
123.                 </div>
124.                 {/* Product Suggestions */}
125.                 {msg.products && msg.products.length > 0 && (
126.                     <div className="mt-3 space-y-2">
127.                         {msg.products.map((product) => {
128.                             // Get the first image from color variants or main images
129.                             const productImage =
130.                                 product.colorVariants?.[0]?.images?.[0] ||
131.                                 product.images?.[0] ||
132.                                 '/placeholder-product.svg';
133.                             return (
134.                                 <div
135.                                     key={product._id}
136.                                     className="bg-[#004643] rounded-lg p-3 shadow-sm hover:shadow-md
137.                                     transition-shadow"
138.                                 >
139.                                     <div className="flex gap-3">
140.                                         {/* Product Image */}
141.                                         <div className="w-20 h-20 rounded-md overflow-hidden">
142.                                             <img
143.                                                 src={productImage}
144.                                                 alt={product.name}
145.                                                 className="w-full h-full object-cover"
146.                                                 onError={(e) => {
147.                                                     e.target.src = '/placeholder-product.svg';
148.                                                 }}
149.                                             />
150.                                         </div>
151.                                         {/* Product Info */}
152.                                         <div className="flex-1 min-w-0">
153.                                             <h4 className="text-sm font-semibold text-[ffffffe]
154.                                                 truncate">
155.                                                 {product.name}
156.                                             </h4>
157.                                             <p className="text-xs text-[#abd1c6] mt-1">
158.                                                 {product.brand} • {product.category}
159.                                             </p>
160.                                             <p className="text-sm font-bold text-[#abd1c6] mt-1">
161.                                                 ${product.price}
162.                                             </p>
163.                                         </div>
164.                                         </div>
165.                                         {/* View Button */}
166.                                         <button
167.                                             onClick={() => handleViewProduct(product._id)}
168.                                             className="w-full mt-3 bg-[#f9bc60] text-[#001e1d] text-sm py-2
169.                                             rounded-md hover:scale-105 transition-transform"
170.                                         >
171.                                             View Product
172.                                         </button>
173.                                     </div>
174.                                 </div>
175.                             );
176.                         })}
177.                     </div>
178.                 )}
179.             </div>

```

```

178.         )))
179.         {isLoading && (
180.             <div className="bg-[#abd1c6] text-[#001e1d] self-start max-w-[80%] rounded-lg
181. p-3">
182.                 <p className="text-sm animate-pulse">Typing...</p>
183.             </div>
184.         )}
185.         {/* Invisible element to scroll to */}
186.         <div ref={messagesEndRef} />
187.     </div>
188.
189.     {/* Scroll to Bottom Button */}
190.     {showScrollButton && (
191.         <button
192.             onClick={scrollToBottom}
193.             className="absolute bottom-18 left-1/2 transform -translate-x-1/2 bg-
194. [ffffffe] border border-[#004643] text-[#004643] rounded-full p-2 shadow-lg hover:scale-110
195. transition-all z-10"
196.             title="Scroll to bottom"
197.             >
198.                 <FaAngleDoubleDown className="w-4 h-4" />
199.             </button>
200.         )}
201.         {/* Input Area */}
202.         <form onSubmit={sendMessage} className="p-3 bg-white border-t border-gray-200
203. flex gap-2 rounded-b-lg">
204.             <input
205.                 type="text"
206.                 value={input}
207.                 onChange={(e) => setInput(e.target.value)}
208.                 placeholder="Ask about boots, gloves..."
209.                 className="flex-1 px-3 py-2 border rounded-md focus:outline-none focus:ring-1
210. focus:ring-[#004643]"
211.                 disabled={isLoading}
212.             />
213.             <button
214.                 type="submit"
215.                 disabled={isLoading}
216.                 className="bg-[#004643] text-[#ffffffe] px-4 py-2 rounded-md hover:scale-110
217. disabled:bg-[#e8e4e6]"
218.                 >
219.                     Send
220.                 </button>
221.             </form>
222.         </div>
223.     )}
224.
225.     {/* Floating Button */}
226.     {isOpen && (
227.         <button
228.             onClick={toggleChat}
229.             className="bg-[#004643] text-[#ffffffe] rounded-full p-4 shadow-lg hover:scale-110
230. transition-transform flex items-center justify-center"
231.             >
232.                 <IoChatbubbleEllipsesSharp/>
233.             </button>
234.         )}
235.     </div>
236. );
237. };
238.
239. export default ChatWidget;
240.
241.

```

6.3 Database Implementation

Schema Validation and Security Hooks (User Model)

The User schema enforces strict validation rules, such as ensuring email addresses are unique and setting default roles to prevent privilege escalation. Crucially, the model implements a Mongoose pre('save') middleware hook to automatically hash user passwords using bcryptjs before they are ever committed to the database, ensuring that plaintext passwords are never exposed.

```
1. const mongoose = require("mongoose");
2. const bcrypt = require("bcryptjs");
3.
4. const addressSchema = new mongoose.Schema({
5.   label: {
6.     type: String,
7.     required: true,
8.     default: "Home"
9.   },
10.  firstName: {
11.    type: String,
12.    required: true
13.  },
14.  lastName: {
15.    type: String,
16.    required: true
17.  },
18.  address: {
19.    type: String,
20.    required: true
21.  },
22.  city: {
23.    type: String,
24.    required: true
25.  },
26.  postalCode: {
27.    type: String,
28.    required: true
29.  },
30.  country: {
31.    type: String,
32.    required: true
33.  },
34.  phone: {
35.    type: String,
36.    required: true
37.  },
38.  isDefault: {
39.    type: Boolean,
40.    default: false
41.  }
42. }, { timestamps: true });
43.
44. const userSchema = new mongoose.Schema({
45.  username: {
46.    type: String,
47.    required: true,
48.    trim: true,
49.  },
50.  email: {
51.    type: String,
52.    required: true,
53.    unique: true,
54.    trim: true,
```

```

55.     match: [/\/S+@\S+\.\S+/, "Please use a valid email address."],
56.   },
57.   password: {
58.     type: String,
59.     required: true,
60.     minLength: 6,
61.   },
62.   role: {
63.     type: String,
64.     enum: ["customer", "admin"],
65.     default: "customer",
66.   },
67.   addresses: [addressSchema]
68. }, { timestamps: true });
69.
70. // Hash password middleware
71. userSchema.pre("save", async function(){
72.   if (!this.isModified("password")) return;
73.   const salt = await bcrypt.genSalt(10);
74.   this.password = await bcrypt.hash(this.password, salt);
75. });
76.
77. // Match user entered password to hashed password in database
78. userSchema.methods.matchPassword = async function(enteredPassword) {
79.   return await bcrypt.compare(enteredPassword, this.password);
80. };
81. module.exports = mongoose.model("User", userSchema);
82.

```

Complex Data Structures and AI Optimization (Product Model)

E-commerce products in the sports domain require highly flexible schemas. The Product model demonstrates how MongoDB handles nested arrays and objects natively (e.g., colorVariants containing nested sizeStock). Furthermore, it incorporates an embedding field to store 768-dimensional float arrays generated by the Gemini AI, which powers the vector search functionality.

```
1. const mongoose = require("mongoose");
2.
3. const productSchema = new mongoose.Schema({
4.   name: {
5.     type: String,
6.     required: true,
7.     trim: true,
8.   },
9.   description: {
10.    type: String,
11.    required: true,
12.  },
13.   originalPrice: {
14.    type: Number,
15.    required: true,
16.    min: 0,
17.  },
18.   discount: {
19.    type: Number,
20.    default: 0,
21.    min: 0,
22.    max: 100,
23.  },
24.   price: {
25.    type: Number,
26.    required: true,
27.    min: 0,
28.  },
29.   countInStock: {
30.    type: Number,
31.    min: 0,
32.    default: 0,
33.  },
34.   sku: {
35.    type: String,
36.    required: true,
37.    unique: true,
38.    trim: true,
39.  },
40.   category: {
41.    type: String,
42.    required: true,
43.    enum: ["BOOTS", "GLOVES", "ACCESSORIES"],
44.  },
45.   accessoryType: {
46.    type: String,
47.    enum: ["Balls", "Socks", "Tape", ""],
48.    default: "",
49.  },
50.   brand: {
51.    type: String,
52.    required: true,
53.  },
54.   age: {
55.    type: [String],
56.    default: ["Adult"],
57.    validate: {
58.      validator: function(arr) {
```

```

59.         return arr.length > 0 && arr.every(val => ["Adult", "Kids"].includes(val));
60.     },
61.     message: 'Age must contain at least one value from: Adult, Kids'
62. },
63. },
64. tags: {
65.     type: [String],
66.     default: [],
67. },
68. // Color variants with their own sizes and stock (for BOOTS and GLOVES)
69. colorVariants: [{
70.     color: {
71.         type: String,
72.         required: true,
73.     },
74.     hexCode: {
75.         type: String,
76.         default: "",
77.     },
78.     images: {
79.         type: [String],
80.         default: [],
81.     },
82.     sizeStock: [{
83.         size: {
84.             type: String,
85.             required: true,
86.         },
87.         stock: {
88.             type: Number,
89.             required: true,
90.             min: 0,
91.             default: 0,
92.         }
93.     }]
94. }],
95. // Stock for ACCESSORIES (no color variants)
96. accessoryStock: {
97.     type: Map,
98.     of: Number,
99.     default: {}
100. },
101. // Boot-specific fields
102. outsole: {
103.     type: String,
104.     enum: ["Firm Ground", "Artificial Grass", "Indoor", ""],
105.     default: "",
106. },
107. material: {
108.     type: String,
109.     enum: ["Leather", "Synthetic", ""],
110.     default: "",
111. },
112. player: {
113.     type: String,
114.     enum: ["None", "Cristiano Ronaldo", "Lionel Messi", "Neymar Jr.", "Kylian Mbappe",
115. "Lamine Yamal", ""],
116.     default: "None",
117. },
118. images: {
119.     type: [String],
120.     default: [],
121.     // Main product images (used as fallback if color variants don't have images)
122. },
123. rating: {
124.     type: Number,
125.     default: 0,
126.     min: 0,

```

```

126.         max: 5,
127.     },
128.     numReviews: {
129.         type: Number,
130.         default: 0,
131.         min: 0,
132.     },
133.     isPublished: {
134.         type: Boolean,
135.         default: true,
136.     },
137.     isBestSeller: {
138.         type: Boolean,
139.         default: false,
140.     },
141.     isNewCollection: {
142.         type: Boolean,
143.         default: false,
144.     },
145.     user: {
146.         type: mongoose.Schema.Types.ObjectId,
147.         ref: "User",
148.         required: false, // Made optional since admin creates products
149.     },
150.     metaTitle: {
151.         type: String,
152.         default: "",
153.     },
154.     metaDescription: {
155.         type: String,
156.         default: "",
157.     },
158.     metaKeywords: {
159.         type: [String],
160.         default: [],
161.     },
162.     dimensions: {
163.         length: { type: Number, min: 0, default: 0 },
164.         width: { type: Number, min: 0, default: 0 },
165.         height: { type: Number, min: 0, default: 0 },
166.     },
167.     weight: {
168.         type: Number,
169.         min: 0,
170.         default: 0,
171.     },
172.     // AI chatbot
173.     embedding: {
174.         type: [Number],
175.         default: []
176.     }
177. }, { timestamps: true });
178.
179. // Auto-calculate price before saving
180. productSchema.pre("save", function() {
181.     if (this.isModified("originalPrice") || this.isModified("discount")) {
182.         this.price = (this.originalPrice - (this.originalPrice * this.discount /
183.         100)).toFixed(2);
184.     }
185.
186.     // Calculate total stock based on category
187.     if (this.category === "ACCESSORIES" && this.accessoryStock) {
188.         // For accessories, sum up accessoryStock values
189.         this.countInStock = Array.from(this.accessoryStock.values()).reduce((sum, val) =>
190.         sum + val, 0);
191.     } else if (this.colorVariants && this.colorVariants.length > 0) {
192.         // For boots and gloves, sum up from color variants
193.         this.countInStock = this.colorVariants.reduce((total, variant) => {

```

```
192.         const variantTotal = variant.sizeStock.reduce((sum, sizeItem) => sum +
sizeItem.stock, 0);
193.         return total + variantTotal;
194.     }, 0);
195. }
196. });
197.
198. // Index for better query performance
199. productSchema.index({ category: 1, brand: 1 });
200. productSchema.index({ tags: 1 });
201. productSchema.index({ price: 1 });
202. productSchema.index({ createdAt: -1 });
203. productSchema.index({ name: "text", description: "text" });
204.
205. module.exports = mongoose.model("Product", productSchema);
206.
207.
```

Relational Mapping in NoSQL (Order Model)

Although MongoDB is non-relational, e-commerce logic heavily relies on relationships between entities. The Order model utilizes Mongoose's ObjectId reference system to link an order document directly back to the User who placed it. This enables the backend to easily "populate" user details when administrators query the order list.

```
1. const mongoose = require("mongoose");
2.
3. const orderItemSchema = new mongoose.Schema({
4.   productId: {
5.     type: mongoose.Schema.Types.ObjectId,
6.     ref: "Product",
7.     required: true,
8.   },
9.   name: {
10.    type: String,
11.    required: true,
12.  },
13.  image: {
14.    type: String,
15.    default: "",
16.  },
17.  price: {
18.    type: Number,
19.    required: true,
20.  },
21.  size: String,
22.  color: String,
23.  quantity: {
24.    type: Number,
25.    required: true,
26.    default: 1,
27.    min: 1
28.  }
29. }, { _id: false });
30.
31. const orderSchema = new mongoose.Schema({
32.   user: {
33.     type: mongoose.Schema.Types.ObjectId,
34.     ref: "User",
35.     required: true,
36.   },
37.   orderItem: [orderItemSchema],
38.   shippingAddress: {
39.     address: { type: String, required: true },
40.     city: { type: String, required: true },
41.     postalCode: { type: String, required: true },
42.     country: { type: String, required: true },
43.     phone: { type: String },
44.   },
45.   paymentMethod: {
46.     type: String,
47.     required: true,
48.   },
49.   totalPrice: {
50.     type: Number,
51.     required: true,
52.     default: 0,
53.   },
54.   isPaid: {
55.     type: Boolean,
56.     default: false,
57.   },
58.   paidAt: {
59.     type: Date,
```

```
60.     },
61.     isDelivered: {
62.       type: Boolean,
63.       default: false,
64.     },
65.     deliveredAt: {
66.       type: Date,
67.     },
68.     paymentStatus: {
69.       type: String,
70.       default: "Pending",
71.     },
72.     status:{
73.       type: String,
74.       enum: [ "Processing", "Shipped", "Delivered", "Cancelled"],
75.       default: "Processing",
76.     },
77.
78.   }, { timestamps: true });
79.
80. module.exports = mongoose.model("Order", orderSchema);
81.
```

6.4 Deployment

Transitioning the TDT-Stadium platform from a local development environment to a live production state was a critical phase of the project. The deployment strategy prioritized Continuous Integration and Continuous Deployment (CI/CD), high availability, and secure management of environment variables. The platform utilizes a distributed cloud architecture, leveraging specialized hosting services for the frontend, backend, database, and media assets.

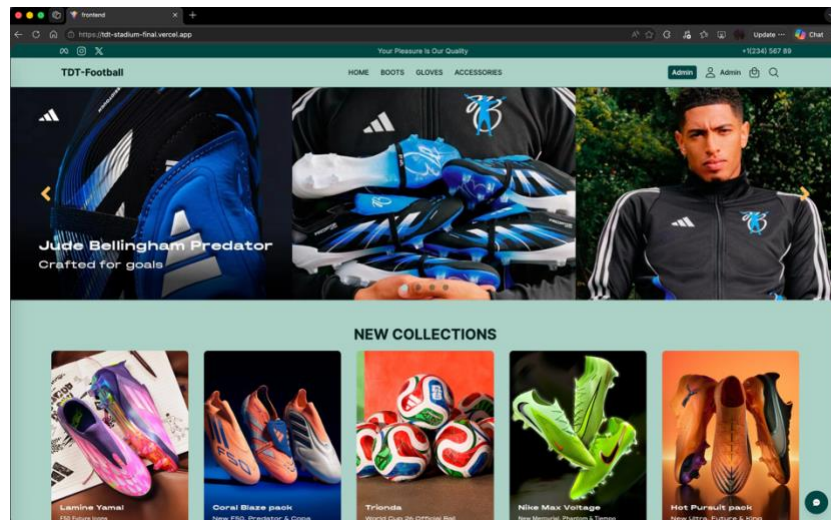
Both the React.js frontend and the Node.js/Express backend were deployed using Vercel, a premier cloud platform optimized for frontend frameworks and serverless functions.

- **Frontend:** The React application (bundled via Vite) is continuously deployed through direct integration with the GitHub repository. Any push to the main branch automatically triggers the `npm run build` command on Vercel's servers, instantly updating the live production site (<https://tdt-stadium-final.vercel.app>).
- **Backend:** Deploying a traditional Express application to Vercel requires converting the server routes into serverless functions. This was achieved by configuring a `vercel.json` file in the backend directory, instructing Vercel to route all incoming HTTP requests to the `server.js` entry point. The backend's Cross-Origin Resource Sharing (CORS) middleware was subsequently updated to accept requests exclusively from the designated Vercel frontend domains, ensuring secure communication between the two separated services.

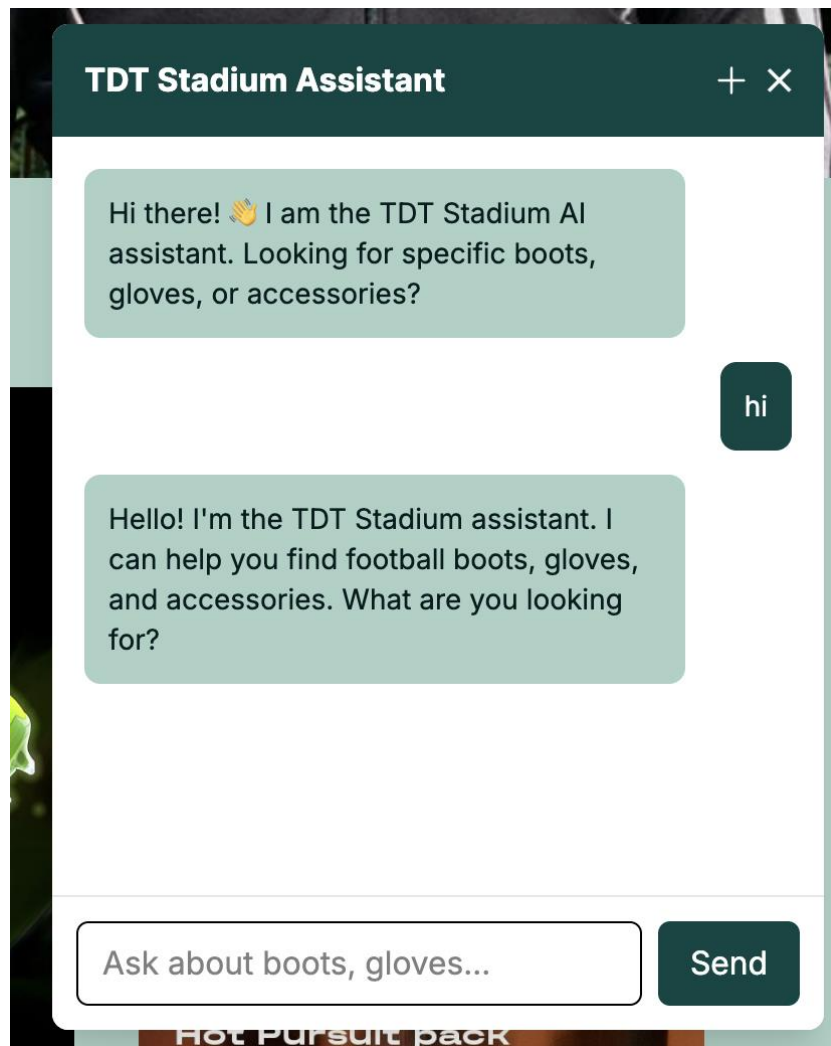
To ensure the persistence and scalability of platform data, local databases were migrated to managed cloud providers:

- **MongoDB Atlas:** The NoSQL database was hosted on MongoDB Atlas. This provided a secure, remote connection string (URI) that the deployed backend uses to read and write application data, completely independent of the Node.js server's lifecycle.
- **Cloudinary:** E-commerce platforms are heavily reliant on high-resolution product imagery. Instead of storing images directly on the Vercel server (which is ephemeral and subject to size limits), Cloudinary was integrated as a dedicated cloud media storage service. The backend's `uploadRoutes.js` intercepts image uploads from the admin dashboard, pipes them securely to Cloudinary using `streamifier`, and returns a highly optimized, global CDN URL (`secure_url`) to be saved in the MongoDB product document

Security in production is paramount. Sensitive credentials required for the application to function such as the `MONGO_URI`, `JWT_SECRET`, `PAYPAL_CLIENT_SECRET`, `GEMINI_API_KEY`, and `CLOUDINARY_API_SECRET` were strictly excluded from version control using `.gitignore`. Instead, these were securely injected into the Vercel Project Settings dashboard. This ensures that third-party API keys remain hidden from public repository viewers while remaining fully accessible to the Node.js `process.env` object during runtime.



DEPLOY TEST 1



DEPLOY TEST 2

6.5 Project Management

The successful delivery of TDT-Stadium within the strictly defined 12-week timeline was made possible through rigorous project management and the consistent application of the Agile Scrum methodology. This section outlines the tools and practices used to monitor progress, manage code quality, and handle the iterative nature of the development lifecycle.

Task Tracking and Sprint Management. In alignment with the Project Plan (Section 4.2), work was divided into focused two-week sprints. A digital Kanban board (via GitHub Projects) served as the "Single Source of Truth" for task status.

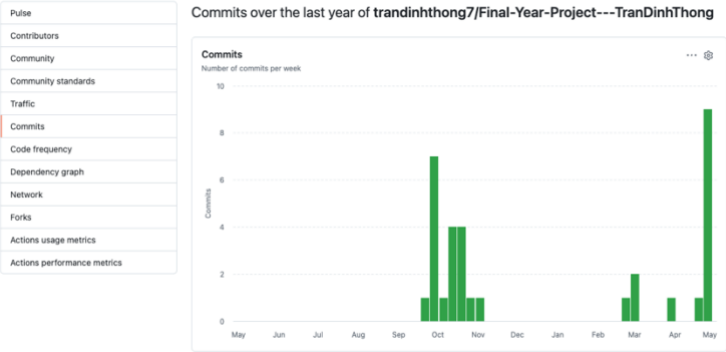
- **Backlog Management:** User stories from the MoSCoW analysis (Section 3.3) were broken down into technical sub-tasks (e.g., "Implement JWT login," "Connect PayPal Sandbox") and assigned to specific sprints.
- **Visual Workflow:** Tasks are moved through three columns: **To Do**, **In Progress**, and **Done**. This allowed for real-time monitoring of the "velocity" of the project and early identification of bottlenecks, particularly during the complex integration of the Gemini AI API in Sprint 4.
-

Version Control and CI/CD Pipeline Git served as the fundamental tool for source code management, ensuring that every change was documented and reversible.

- **Commit Discipline:** A "Commit Early, Commit Often" strategy was adopted. Each commit was accompanied by a descriptive message (e.g., "feat: add vector search for product recommendations") to provide a clear audit trail of the project's evolution.
- **GitHub Integration:** The central repository on GitHub acted as the trigger for the project's CI/CD pipeline. As detailed in the Deployment section (6.4), successful pushes to the main branch automatically initiate production builds on Vercel, ensuring that the live application always reflects the latest stable code.

Quality Assurance and Environment Management To maintain a professional standard of development, specific configuration files were implemented to manage the environment and code quality:

- **Exclusion Policy:** A `.gitignore` file was strictly maintained to prevent sensitive configuration files (`.env`) and heavy dependency folders (`node_modules`) from being uploaded to the public repository, adhering to professional security standards.
- **Linting:** ESLint was configured within the frontend to enforce a consistent coding style and catch potential bugs during the coding phase, significantly reducing the time required for the final testing phase.



GITHUB COMMIT 1

95%	trandinhthong7 authored yesterday ✓ 1/1	Verified ec050ec
95%	trandinhthong7 committed yesterday	c36781a
Commits on Apr 22, 2026		
admin	trandinhthong7 committed last week	29a932e
Commits on Mar 29, 2026		
authentication	trandinhthong7 committed on Mar 29	25d683c
Commits on Mar 6, 2026		
footer	trandinhthong7 committed on Mar 5	4afa726
Commits on Mar 2, 2026		
fix most popular again again and again	trandinhthong7 committed on Mar 2	661c035
Commits on Feb 26, 2026		
1	trandinhthong7 committed on Feb 26	55ab49e
Commits on Nov 7, 2025		
product detail and sonner	trandinhthong7 committed on Nov 2, 2025	12ce851
Commits on Oct 26, 2025		

GITHUB COMMIT 2

main	All users	All time
Commits on Apr 30, 2026		
100,2%	trandinhthong7 committed 2 hours ago ✓ 10/10	a9ed113
Commits on Apr 29, 2026		
100,1%	trandinhthong7 committed 10 hours ago ✓ 10/10	97b60e
100%	trandinhthong7 committed 10 hours ago ✓ 10/10	5ea3679
Commits on Apr 28, 2026		
95%	trandinhthong7 committed yesterday ✓ 6/6	777d93
git add	trandinhthong7 committed yesterday ✗ 1/3	02ecac0
test deploy	trandinhthong7 committed yesterday	89f2628
95%	trandinhthong7 authored yesterday ✗ 1/3	Verified 6a46a3c
m	trandinhthong7 committed yesterday	b96ca2d
95%	trandinhthong7 authored yesterday ✓ 1/1	Verified ec050ec
95%	trandinhthong7 committed yesterday	c36781a

GITHUB COMMIT 3

7. Testing

7.1 Testing Strategy

The testing strategy for TDT-Stadium was designed to be comprehensive, ensuring that every layer of the MERN stack—from the database schemas to the React UI—operates reliably under various conditions. The strategy utilized a "V-Model" inspired approach, where testing levels corresponded directly to the development phases.

The following four levels of testing were implemented:

Unit Testing:

- **Focus:** Testing individual functions and components in isolation.
- **Application:** In the backend, this involved testing helper functions such as the `calculateTotalPrice` logic in `cartRoutes.js` and the `matchPassword` method in the `User` model to ensure mathematical and cryptographic accuracy before integration.

Integration Testing:

- **Focus:** Verifying that different modules and external services work together correctly.
- **Application:** Significant emphasis was placed on the communication between the Node.js server and the PayPal Developer API. This ensured that order IDs generated by the backend were correctly recognized by PayPal's servers and that the subsequent "Capture" callback correctly triggered order finalization in the database.

System Testing (Black-Box Testing):

- **Focus:** Testing the complete, integrated system against the functional requirements.
- **Application:** This involved end-to-end testing of the "Critical Path"—from user registration and product filtering to adding items to a cart and completing a checkout. This phase ensured that the Redux global state stayed synchronized across different page navigations and that the AI Chatbot provided relevant product recommendations based on real database entries.

User Acceptance Testing (UAT):

- **Focus:** Determining if the system meets the needs of the end-users and is ready for production.
- **Application:** Representative users (matching the personas of athletes and parents defined in Section 3.2) were asked to perform specific tasks on the live Vercel deployment. Their feedback was used to validate the usability of the specialized technical specification filters and the clarity of the administrative dashboard.

7.2 Test Cases and Results

Present a table with sample test cases. Include columns for Test ID, Test Description, Steps, Expected Result, Actual Result, and Pass/Fail.

Test ID	Test Description	Steps	Expected Result	Actual Result	Pass/Fail
TC_01	User Registration	1. Navigate to /register. 2. Enter valid username, email, and password. 3. Click "Register".	User is created in MongoDB with a hashed password; a JWT is returned.	User created; JWT received in local storage.	Pass
TC_02	Secure Login	1. Navigate to /login. 2. Enter correct credentials. 3. Click "Login".	System validates credentials via bcrypt, returns JWT, and redirects to home.	Login successful; user profile state updated in Redux.	Pass
TC_03	Product Filtering	1. Go to "Boots" page. 2. Select "Nike" brand and "AG" outsole filters.	Only Nike boots with an "AG" outsole type are displayed.	Correct products filtered based on productRoutes.js logic.	Pass
TC_04	Cart Persistence	1. Add an item to the cart. 2. Refresh the browser.	The shopping cart count and items remain visible.	Cart items persisted via Redux-Persist/Local Storage.	Pass
TC_05	AI Chatbot Accuracy	1. Open Chat Widget. 2. Type "I need boots for artificial grass."	AI identifies "BOOTS" category and "AG" outsole, recommending relevant products.	AI returned 3 AG boots with correct technical details.	Pass
TC_06	PayPal Checkout	1. Proceed to checkout. 2. Log in to PayPal Sandbox. 3. Approve payment.	PayPal order is captured; Backend creates a new Order document.	Order created; payment status marked as "Paid" in DB.	Pass
TC_07	Admin Dashboard Access	1. Log in with a standard "user" account.	Access is denied; user is redirected to home page.	authMiddleware.js successfully blocked the unauthorized route.	Pass

Test ID	Test Description	Steps	Expected Result	Actual Result	Pass/Fail
		2. Attempt to access /admin/products.			
TC_08	Admin CRUD Operation	1. As Admin, create a new product. 2. Save and check storefront.	New product appears in the "New Collection" section of the storefront.	Product successfully saved to MongoDB and rendered in UI.	Pass
TC_09	Responsive Design	1. Open site on a mobile device (375px width). 2. Check Navigation and Product Grid.	Navigation collapses into a hamburger menu; grid adjusts to a single column.	UI elements stacked correctly for mobile usability.	Pass
TC_10	Address Management	1. Add a new shipping address in profile. 2. Set it as "Default".	Other addresses are automatically marked as isDefault: false.	addressRoutes.js correctly toggled the default status flags.	Pass

8. Legal, Social, Ethical, and Professional Issues

The development and deployment of the TDT-Stadium platform are governed by a range of legal, social, ethical, and professional considerations. Adhering to these standards ensures that the platform is not only technically sound but also responsible and trustworthy for its users and the broader community.

Legal Issues

The primary legal concern for TDT-Stadium is Data Privacy and Protection. Since the system collects and stores Personally Identifiable Information (PII) such as names, email addresses, and shipping details in the User and Address collections, it must comply with the General Data Protection Regulation (GDPR).

- **Data Minimization:** The system only requests data essential for account creation and order fulfillment.
- **Intellectual Property (IP):** All third-party libraries used (e.g., React, Express, Mongoose) are open-source and used in accordance with their respective MIT or Apache licenses.
- **Financial Compliance:** By integrating the PayPal Developer API, TDT-Stadium avoids storing sensitive credit card numbers on its local servers, thereby reducing the burden of PCI-DSS compliance and ensuring secure, legally-compliant transactions.

Social Issues

TDT-Stadium aims to have a positive social impact by improving the accessibility of high-quality sports equipment for amateur and semi-professional athletes.

- **Digital Inclusion:** The platform's mobile-first, responsive design ensures that users who rely solely on mobile devices (identified as 85% of the target audience in Section 3.2) are not excluded from the service.
- **Community Support:** By providing detailed technical specifications (e.g., ground type and latex grip), the platform empowers users with the knowledge to select equipment that prevents injuries, such as using "AG" boots on artificial turf to avoid knee strain.
- **Accessibility:** The UI follows standard web accessibility guidelines, using semantic HTML and high-contrast color schemes to ensure usability for individuals with visual impairments.

Ethical Issues

Ethical considerations were integrated into the core architecture of the system, particularly regarding data security and artificial intelligence.

- **Security Ethics:** It is ethically imperative to protect user credentials. TDT-Stadium implements bcryptjs to hash passwords, ensuring that even in the event of a database breach, user passwords remain unreadable.
- **AI Transparency and Bias:** The Google Gemini AI integration for the chatbot is designed to provide objective, product-based information. Great care was taken to ensure the AI only recommends products based on technical specifications rather than biased or sponsored rankings.
- **Transparency:** Users are clearly informed when they are interacting with an automated AI system rather than a human agent.

Professional Issues

As a computing professional, the project was developed in accordance with the BCS (British Computer Society) Code of Conduct.

- **Professional Standards:** The project utilizes industry-standard tools (Git, GitHub, Vercel) and follows a structured Agile methodology to ensure the delivery of high-quality software.
- **Duty to Relevant Authority:** The codebase is thoroughly documented, and all source code is stored in a structured repository, allowing for transparency and auditability by academic supervisors.
- **Competence:** The developer maintained professional competence by strictly adhering to the MERN stack best practices, including centralized state management (Redux) and secure API routing.

9. Conclusion and Critical Reflection

9.1 Summary of Achievements

The primary aim of the TDT-Stadium project—to develop a specialized e-commerce platform for football equipment—has been successfully realized. The final system demonstrates a high level of technical integration, moving beyond a standard shopping site to provide a technical, data-driven experience for athletes.

The following table summarizes the achievement of the specific project objectives defined in Section 1.4:

Objective	Status	Achievement Details
1. Responsive Frontend	Completed	Developed a high-performance Single-Page Application (SPA) using React.js and Vite, ensuring a mobile-first experience for athletes on the go.
2. Robust Backend & DB	Completed	Implemented a scalable MERN-stack architecture with a Node.js/Express API and a flexible MongoDB database schema to handle diverse product data.
3. Technical Filtering	Completed	Engineered a complex dynamic filtering system in productRoutes.js that allows users to sort gear by specific attributes like "Latex Type" and "Outsole".
4. Secure Payments	Completed	Successfully integrated the PayPal Developer API to facilitate secure, server-side verified transactions, achieving a critical requirement for e-commerce trust.
5. AI-Powered Chat	Completed	Integrated the Google Gemini API to provide an intelligent chatbot capable of natural language product recommendations and technical support.

Technical Gap Closure: The project successfully addressed the "information frustration" identified in the market analysis (Section 3.2). By providing deep technical specifications in the product catalog, TDT-Stadium offers a level of detail that generic sports retailers lack.

Security Integration: The implementation of JWT-based authentication and bcrypt password hashing ensures that the platform meets modern professional security standards for user data protection.

Innovative Feature Set: The inclusion of AI-driven vector search within the Chatbot represents a sophisticated use of modern generative AI technology, enhancing the user's ability to find specialized gear through conversational prompts.

9.2 Critical Project Evaluation

The transition from a monolithic local setup to a decoupled cloud architecture (Vercel + Render/Vercel) provided a significant learning curve. While the MERN stack proved highly effective for rapid development, the "Serverless" nature of the Vercel deployment introduced unexpected "Cold Start" latencies. This means that after periods of inactivity, the backend API can take several seconds to "wake up," which slightly conflicts with the Non-functional Requirement NF_01 regarding a 2.5-second load time.

Implementing MongoDB Vector Search for the AI Chatbot was the most complex technical achievement. However, a critical evaluation reveals that the accuracy of the AI is entirely dependent on the quality of the description and technical specs strings in the database. If a product lacks detailed metadata, the AI occasionally provides "hallucinated" suggestions or generic advice rather than specific gear recommendations.

Limitations

Image Handling: Due to the ephemeral nature of free-tier cloud hosting, local image storage was impossible. While the Cloudinary integration solved this, the free tier limits the number of high-resolution transformations, which could hinder a large-scale catalog.

Payment Scope: The PayPal integration is currently restricted to the "Sandbox" environment. While this proves the technical logic, real-world deployment would require rigorous business verification and handling of various international tax jurisdictions.

Real-time Inventory: Currently, stock levels are updated manually by the admin. The system lacks a real-time "Websocket" connection to push stock updates to all active users simultaneously, meaning a user could theoretically add an item to their cart just as it sells out to another user.

The Agile Scrum methodology was vital for staying on track. However, "Scope Creep" was an initial issue; the original plan included a user review system and multilingual support. These were moved to the "Won't Have" category of the MoSCoW list during Sprint 2 to prioritize the stability of the core AI and Payment features, proving that disciplined prioritization is essential for meeting fixed deadlines.

9.3 Personal Reflection and Key Learnings

The completion of the TDT-Stadium project has been a transformative experience, marking a significant milestone in my development as a software engineer. This section reflects on the journey from conceptualization to deployment, highlighting the key technical and professional skills acquired throughout the process.

Before initiating this project, my experience with full-stack development was primarily theoretical. Building TDT-Stadium forced me to bridge the gap between academic concepts and production-ready code.

The MERN Stack & Vite: Mastering the MERN (MongoDB, Express, React, Node.js) stack provided a deep understanding of asynchronous JavaScript and the importance of a unified language across the stack. Transitioning to Vite was a pivotal learning point, demonstrating how modern tooling can significantly improve developer experience and application performance.

State Management with Redux: Implementing Redux Toolkit taught me the importance of a "Single Source of Truth." Managing complex states—such as a persistent shopping cart that merges guest and user data—was one of the most challenging yet rewarding technical hurdles.

AI Integration: Working with the Google Gemini API and Vector Search was a leap into modern software trends. I learned how to transform natural language into mathematical vectors to provide a more intuitive search experience, moving beyond simple keyword matching.

The project was as much a lesson in discipline as it was in coding.

Agile Resilience: Adhering to the Agile Scrum methodology taught me the value of iterative progress. By breaking the massive scope of an e-commerce platform into two-week sprints, I was able to maintain momentum and manage "scope creep" effectively.

Problem-Solving under Pressure: Encountering deployment issues on Vercel and handling CORS errors between the frontend and backend required a systematic approach to debugging. I learned that documentation and community forums (like Stack Overflow and official API docs) are as essential to a developer as the code editor itself.

Ethical Responsibility: Designing the system with GDPR compliance and secure password hashing in mind instilled a sense of professional ethics. I now view security not as an "add-on" feature, but as a foundational requirement of any software system.

Ultimately, TDT-Stadium has equipped me with a robust portfolio and the confidence to tackle complex, real-world computing challenges. The journey has reinforced my passion for creating user-centric technology that solves specific community needs—in this case, the specialized needs of the football community.

9.4 Future Work

While the current version of TDT-Stadium successfully delivers a specialized e-commerce experience with AI integration, the platform has been designed with a modular architecture to allow for significant future expansion. The following features are identified as the primary focus for the next development iteration:

Integrated Stadium Booking System: To transform the platform into a comprehensive football hub, a booking module will be implemented. This will allow users like Coach Minh or An (Section 3.2) to check real-time availability of local football pitches and book sessions directly through the app, utilizing a calendar interface and automated scheduling logic.

Academy and Training Classes: A dedicated section for football academies will be introduced, enabling parents like Mrs. Linh to browse, register, and pay for youth training programs. This module will support recurring subscriptions and attendance tracking for coaches.

Direct Customer Service Integration: To complement the current AI Chatbot, a live "Human-in-the-Loop" customer service feature will be added. This will allow users to escalate complex technical queries to professional staff via a real-time WebSocket connection, ensuring a higher level of personalized support.

Enhanced FAQ and Knowledge Base: A structured, searchable FAQ section will be developed. This will utilize the existing Gemini AI architecture to automatically suggest relevant articles based on the user's browsing behavior, such as size conversion guides for different brands or maintenance tips for goalkeeper gloves.

Peer-to-Peer Review and Community System: To build further consumer trust, a verified purchase review system will be implemented. This will allow users to upload photos of their gear in action and provide "Performance Ratings," which can then be used to further train the AI recommendation engine.

9.5 Conclusion

The TDT-Stadium project has successfully culminated in the development of a sophisticated, specialized e-commerce ecosystem designed to meet the distinct technical and functional needs of the football community. By shifting the focus from generic sports retail to a technically informed platform, the project has effectively addressed the information gap identified during the market research phase, providing athletes and parents with the specialized data necessary to make informed equipment choices.

From a technical perspective, the project serves as a comprehensive demonstration of full-stack engineering proficiency using the MERN stack. The successful implementation of an AI-powered chatbot utilizing vector search for semantic product matching, alongside a secure, server-side verified PayPal payment gateway, elevates the platform beyond a standard prototype to a professionally aligned application. Furthermore, the project's disciplined adherence to the Agile Scrum methodology ensured that all "Must-Have" requirements were delivered within the strict 12-week timeline.

In conclusion, TDT-Stadium represents the successful integration of modern software development practices, user-centric design, and innovative AI technologies. It fulfills all the requirements of the BSc (Hons) Computing final year project and provides a robust, scalable foundation for future enhancements—such as stadium bookings and academy services—aimed at supporting the broader grassroots football landscape.

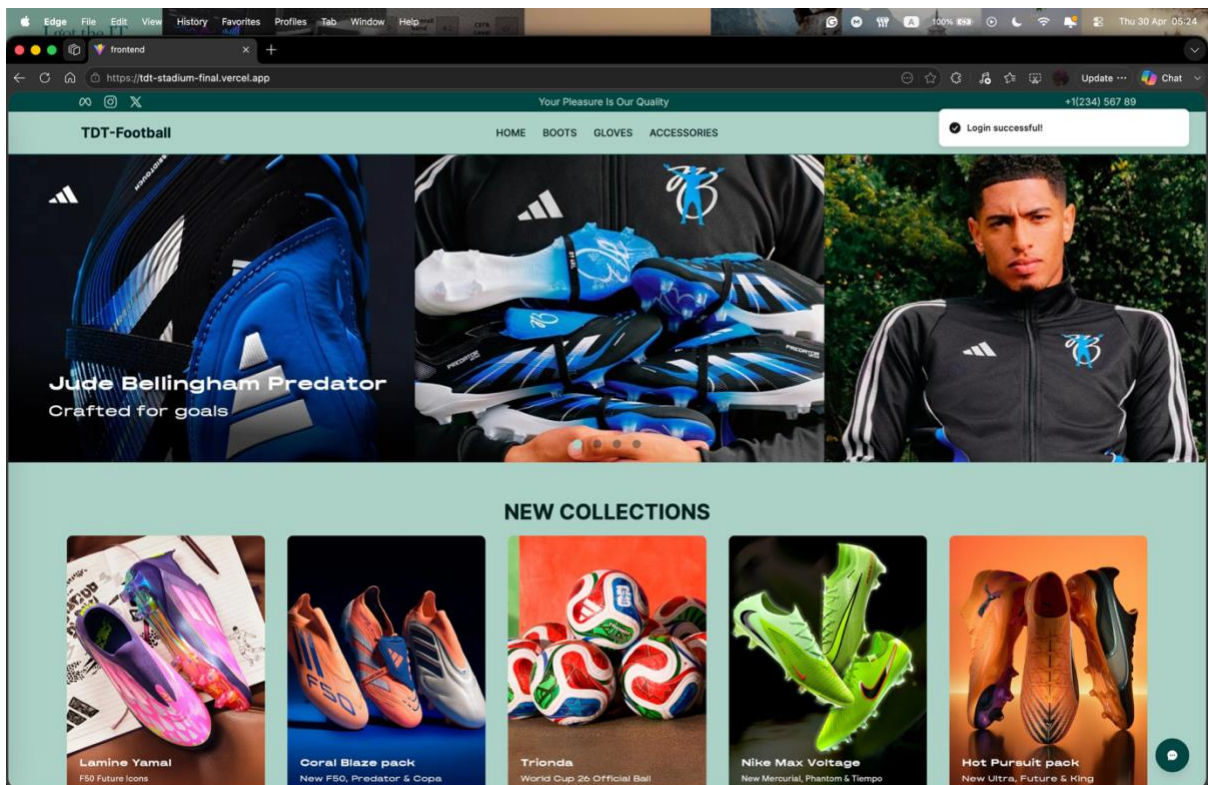
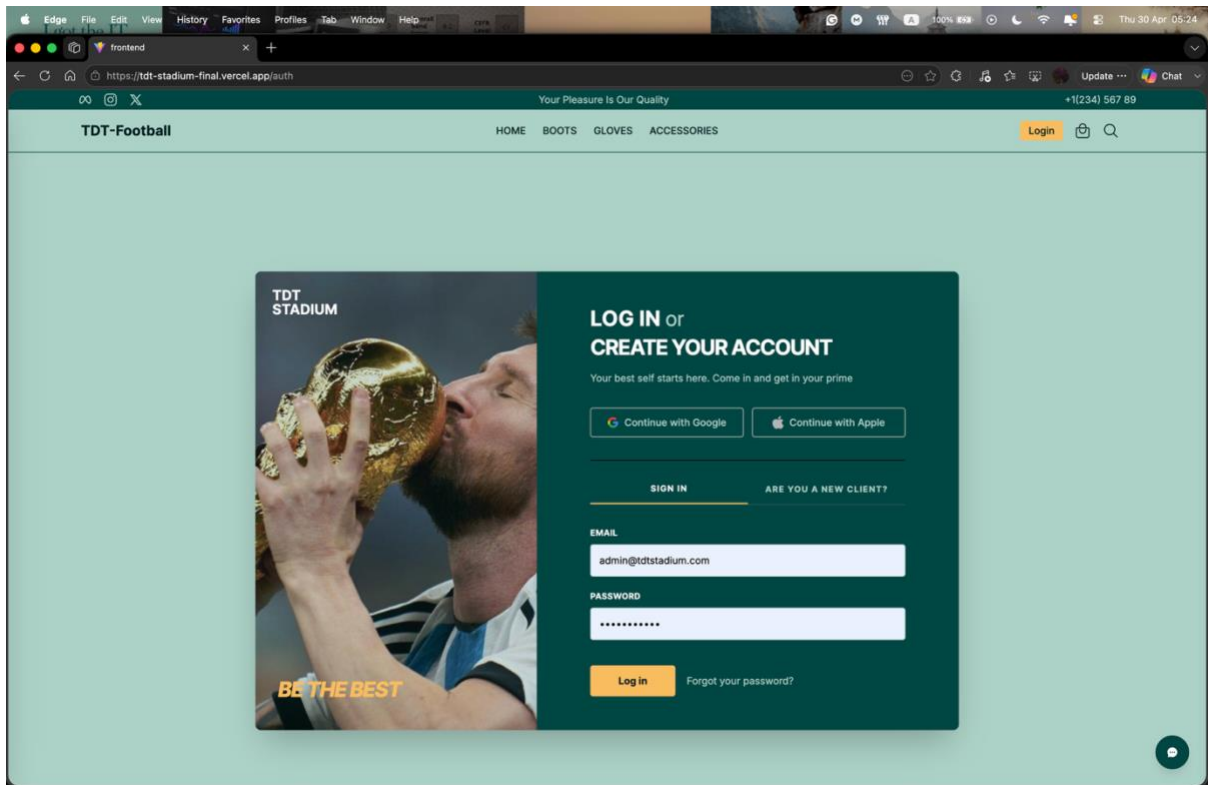
10. References

- Chacon, S. and Straub, B. (2014). Pro Git. 2nd ed. New York: Apress.
- Google for Developers. (2026). Gemini API Documentation. [online] Available at: <https://ai.google.dev/docs> [Accessed 16 Feb. 2026].
- IETF. (2015). RFC 7519: JSON Web Token (JWT). [online] Available at: <https://datatracker.ietf.org/doc/html/rfc7519> [Accessed 25 Feb. 2026].
- Meta Platforms, Inc. (2026). React: The library for web and native user interfaces. [online] Available at: <https://react.dev/> [Accessed 10 Mar. 2026].
- MongoDB, Inc. (2026). MongoDB Documentation. [online] Available at: <https://www.mongodb.com/docs/> [Accessed 26 Feb. 2026].
- OpenJS Foundation. (2026). Node.js Documentation. [online] Available at: <https://nodejs.org/en/docs/> [Accessed 24 Feb. 2026].
- PayPal Developer. (2026). PayPal Checkout API Integration. [online] Available at: <https://developer.paypal.com/docs/checkout/> [Accessed 12 Mar. 2026].
- Redux. (2026). Redux Toolkit: The official, opinionated, batteries-included toolset for efficient Redux development. [online] Available at: <https://redux-toolkit.js.org/> [Accessed 12 Mar. 2026].
- Schwaber, K. and Sutherland, J. (2020). The Scrum Guide. [online] Available at: <https://scrumguides.org/scrum-guide.html> [Accessed 05 Feb. 2026].
- Vercel. (2026). Vercel Documentation: Deploying Node.js and React Applications. [online] Available at: <https://vercel.com/docs> [Accessed 14 Apr. 2026].
- Vite. (2026). Vite: Next Generation Frontend Tooling. [online] Available at: <https://vitejs.dev/> [Accessed 10 Mar. 2026].

11. Appendix A – Project Proposal

https://docs.google.com/document/d/1X3lyppsdxawRSbO99sTpDdq7TF3rNybe/edit?usp=drive_link&ouid=100459270302958033379&rtpof=true&sd=true

12. Appendix B – Images for your final product



Edge File Edit View History Favorites Profiles Tab Window Help 100% \$58 Thu 30 Apr 05:24

frontend x +

https://tdt-stadium-final.vercel.app

MOST POPULAR

BOOTS GLOVES ACCESSORIES



DEAL
NIKE PHANTOM 6 LOW PRO FG
\$216 ~~\$270~~ -20%



DEAL
THUONG DINH PRO BLACK
\$56 ~~\$80~~ -30%



DEAL BEST SELLER
NIKE AIR ZOOM MERCURIAL VAPOR
\$237.8 ~~\$290~~ -18%



DEAL NEW
PUMA ZUKUNFTIGE 8 PRO FG/AG
\$202.8 ~~\$260~~ -22%
39 40 41 42 43



DEAL
UHLSPORT CYBERTE
\$63.75 ~~\$85~~ -25%

BEST SELLER




Adidas F50 Elite AG Purple
NEW BEST SELLER
\$242.25 ~~\$285~~ -15%
Lightning-fast boots with Sprintframe 360 outsole for explosive acceleration on artificial grass. Features Fiberskin upper for advanced ball control.

https://tdt-stadium-final.vercel.app/product/69f19cc10c4f08487fc914f3

Edge File Edit View History Favorites Profiles Tab Window Help 100% \$58 Thu 30 Apr 05:25

frontend x +

https://tdt-stadium-final.vercel.app



39 40 41 42 43
Quantity: 1
ADD TO CART

YOU MAY ALSO LIKE



NEW BEST SELLER
ADIDAS F50 ELITE YAMAL UNITY EDITION
\$272.8



NEW BEST SELLER
ADIDAS F50 ELITE LL FG PURPLE
\$265.5
39 40 41 42 43

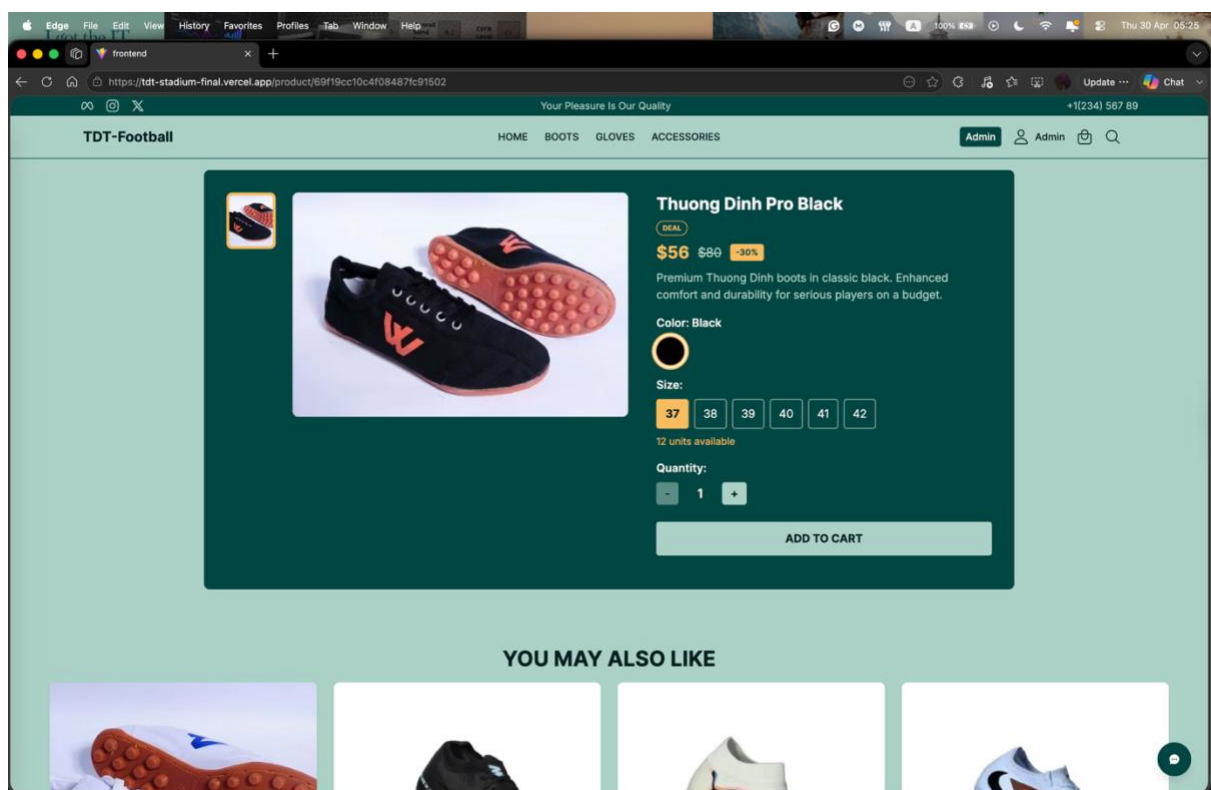
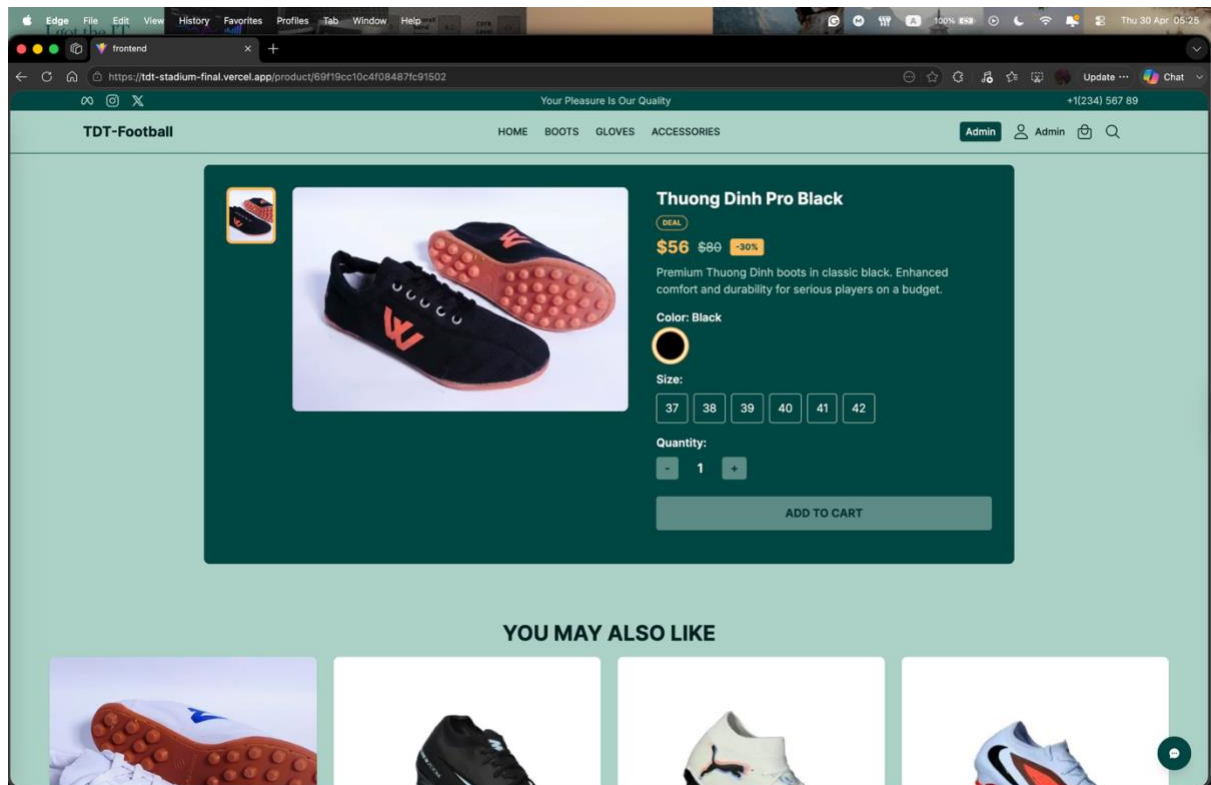


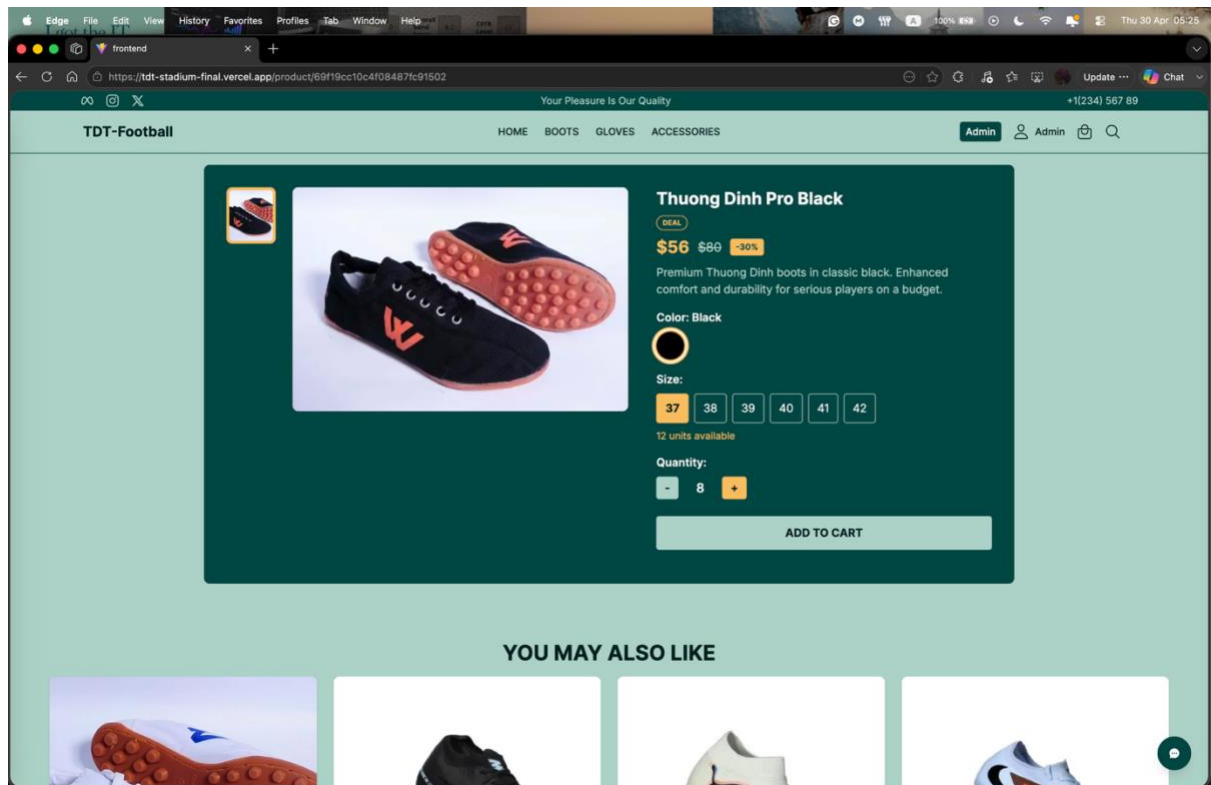
DEAL
NIKE PHANTOM 6 LOW PRO FG
\$216 ~~\$270~~

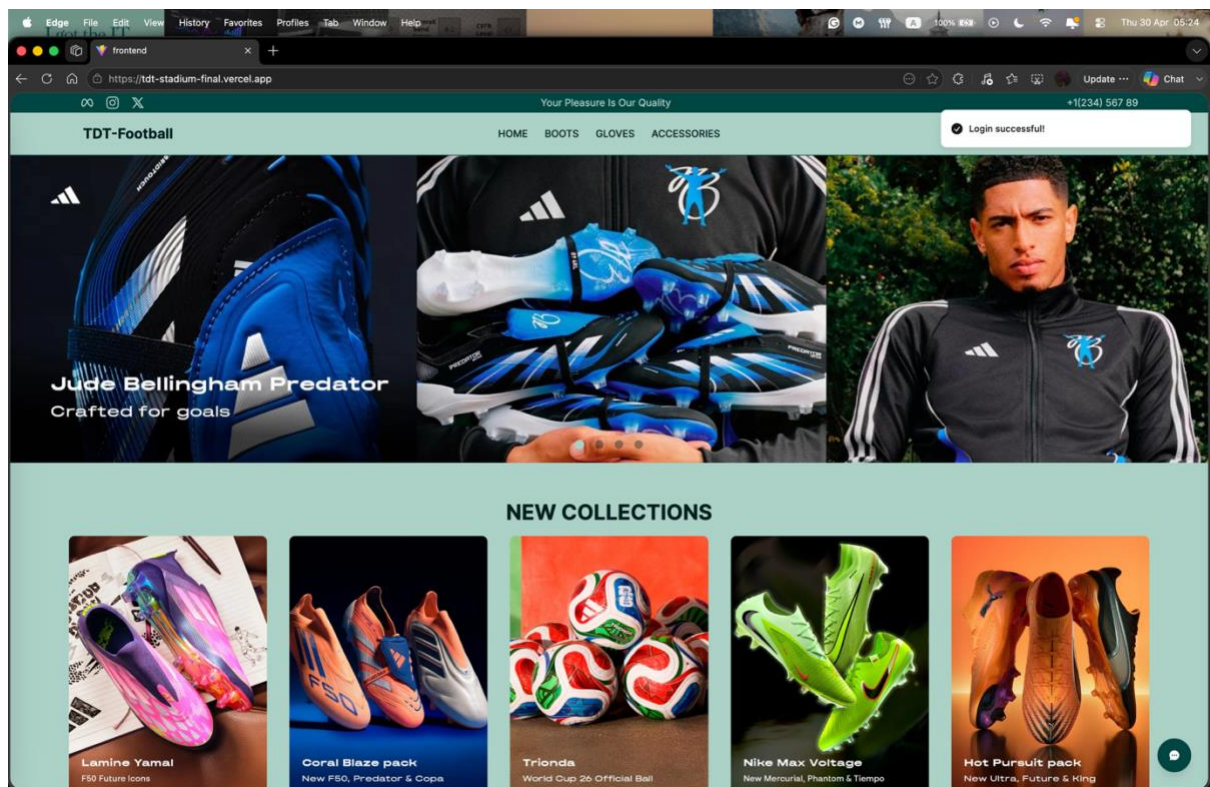
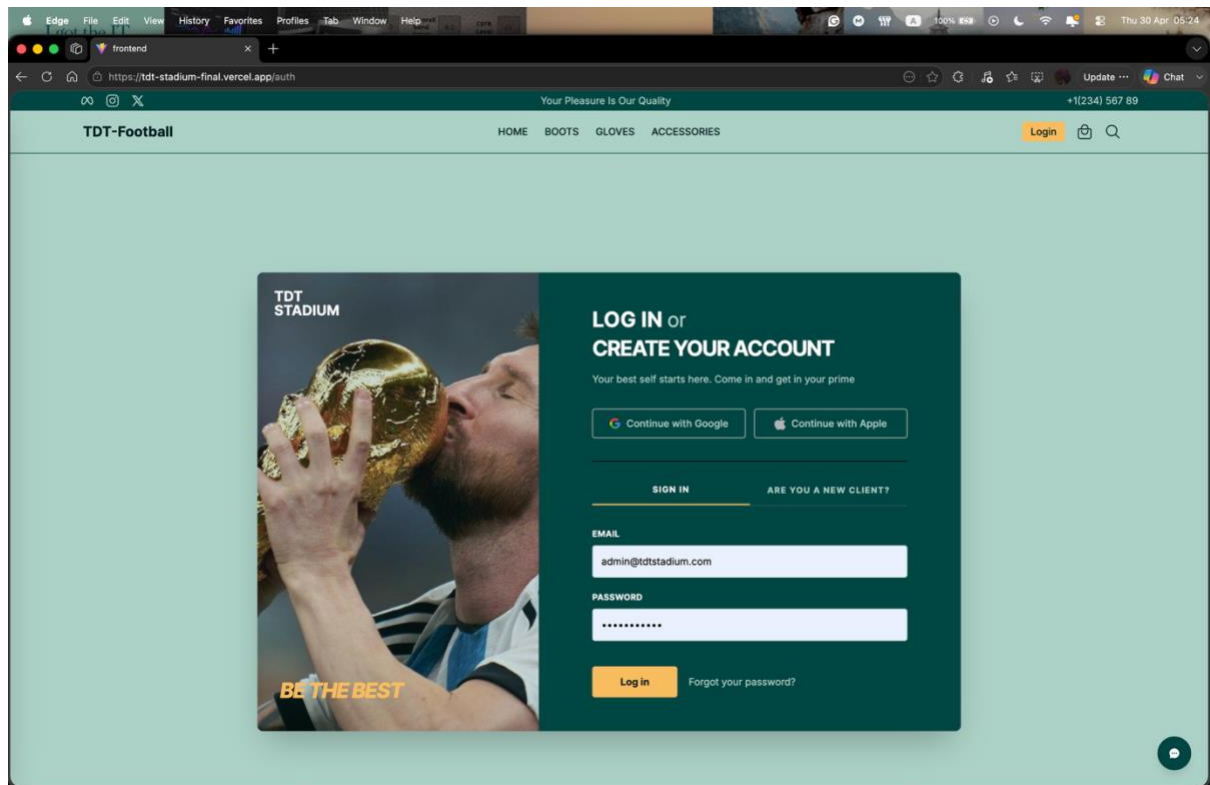


DEAL BEST SELLER
NIKE AIR ZOOM MERCURIAL VAPOR
\$237.8 ~~\$290~~

https://tdt-stadium-final.vercel.app/product/69f19cc10c4f08487fc914d7







Edge File Edit View History Favorites Profiles Tab Window Help 100% 858 Thu 30 Apr 05:24

frontend x +

https://tdt-stadium-final.vercel.app

MOST POPULAR

BOOTS GLOVES ACCESSORIES

DEAL
NIKE PHANTOM 6 LOW PRO FG
\$216 ~~\$270~~ -20%

DEAL
THUONG DINH PRO BLACK
\$56 ~~\$80~~ -30%

DEAL BEST SELLER
NIKE AIR ZOOM MERCURIAL VAPOR
\$237.8 ~~\$290~~ -18%

DEAL NEW
PUMA ZUKUNFTIGE 8 PRO FG/AG
\$202.8 ~~\$260~~ -22%
39 40 41 42 43

DEAL
UHLSPORT CYBERTE
\$63.75 ~~\$85~~ -25%

BEST SELLER

NEW BEST SELLER
Adidas F50 Elite AG Purple
\$242.25 ~~\$285~~ -15%
Lightning-fast boots with Sprintframe 360 outsole for explosive acceleration on artificial grass. Features Fiberskin upper for enhanced ball control.

https://tdt-stadium-final.vercel.app/product/69f19cc10c4f08487c914f3

Edge File Edit View History Favorites Profiles Tab Window Help 100% 858 Thu 30 Apr 05:25

frontend x +

https://tdt-stadium-final.vercel.app

39 40 41 42 43
Quantity:
- 1 +
ADD TO CART

YOU MAY ALSO LIKE

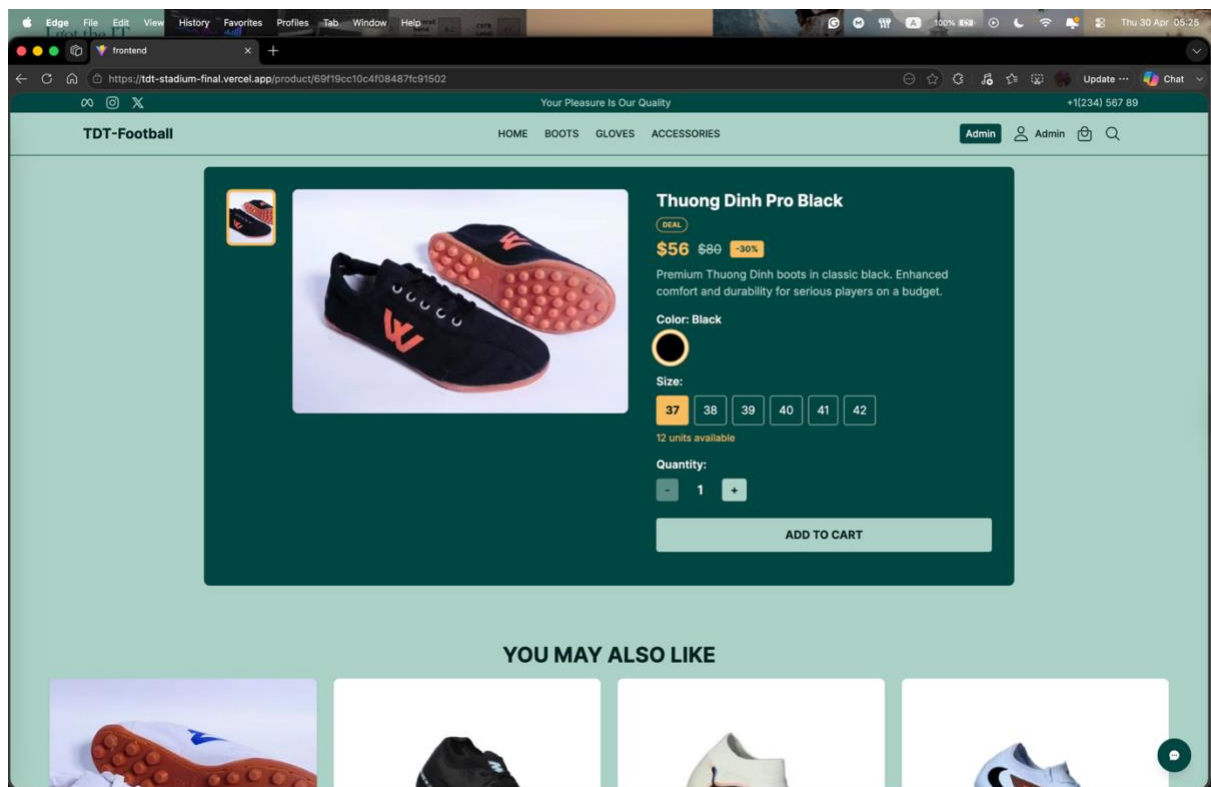
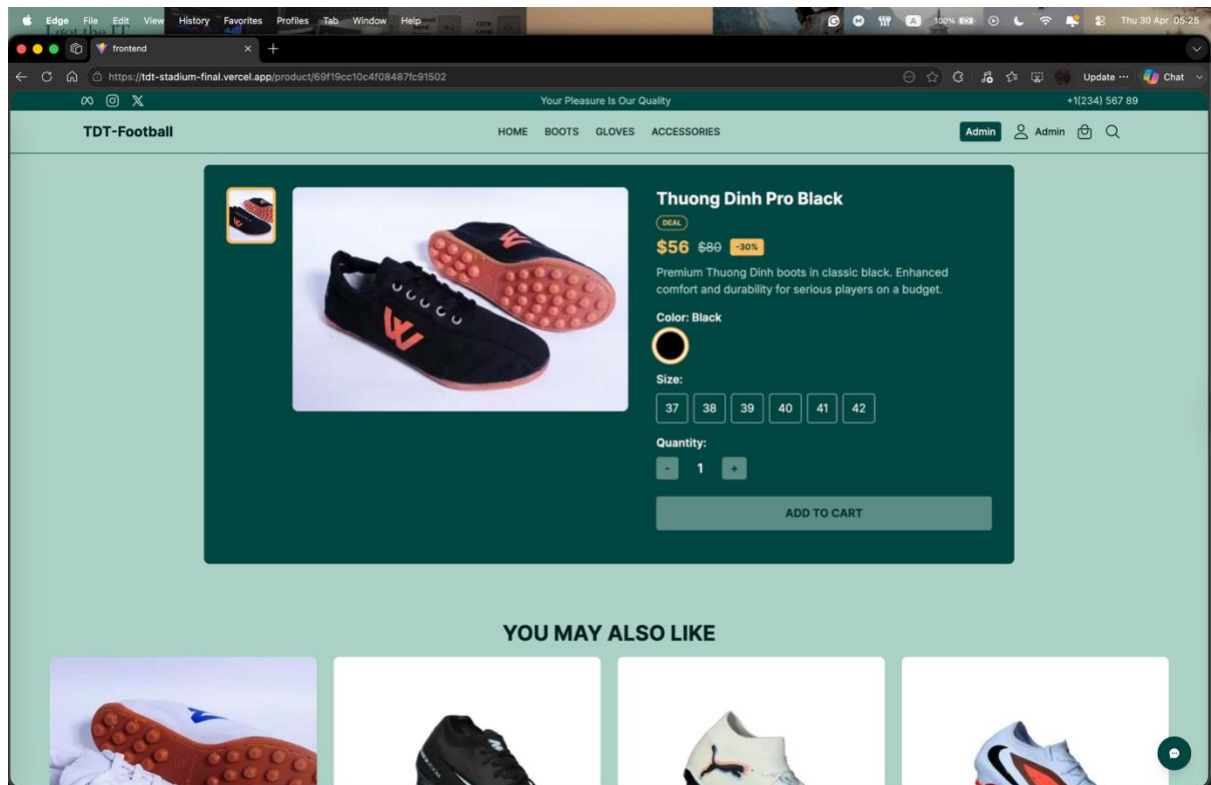
NEW BEST SELLER
ADIDAS F50 ELITE YAMAL UNITY EDITION
\$272.8

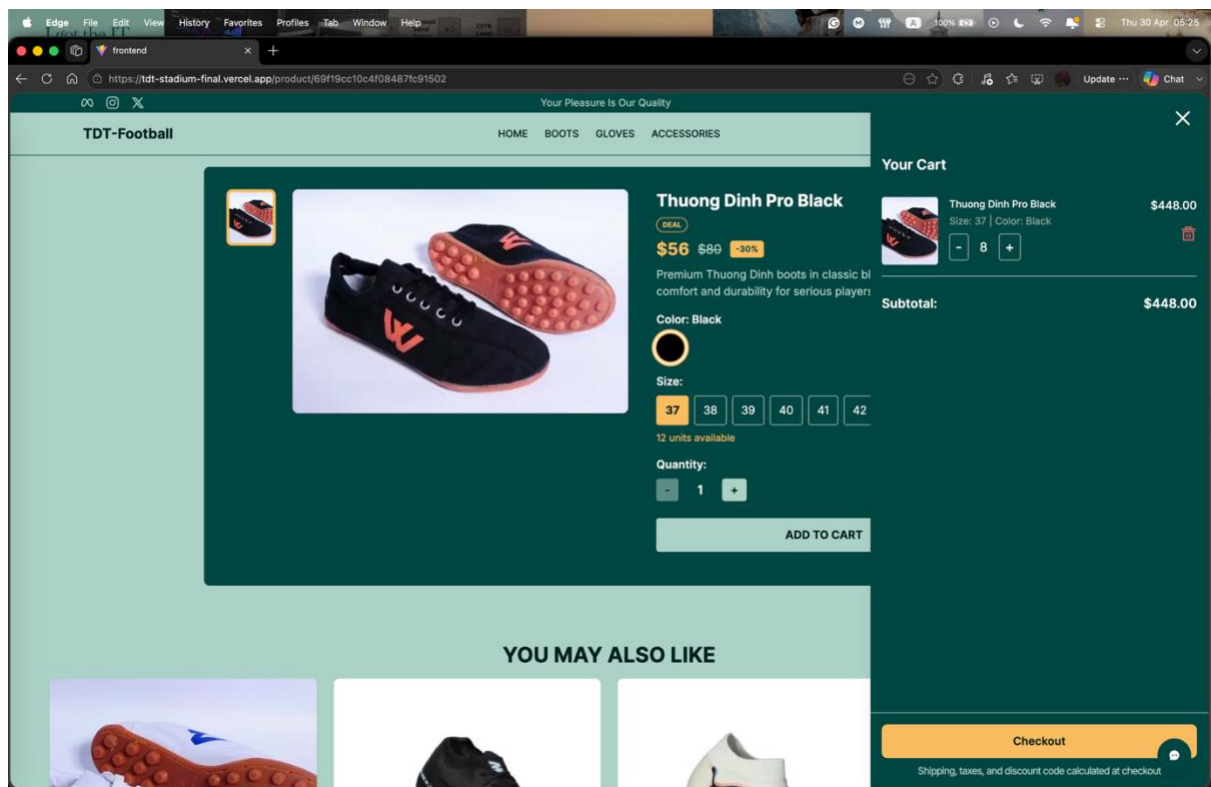
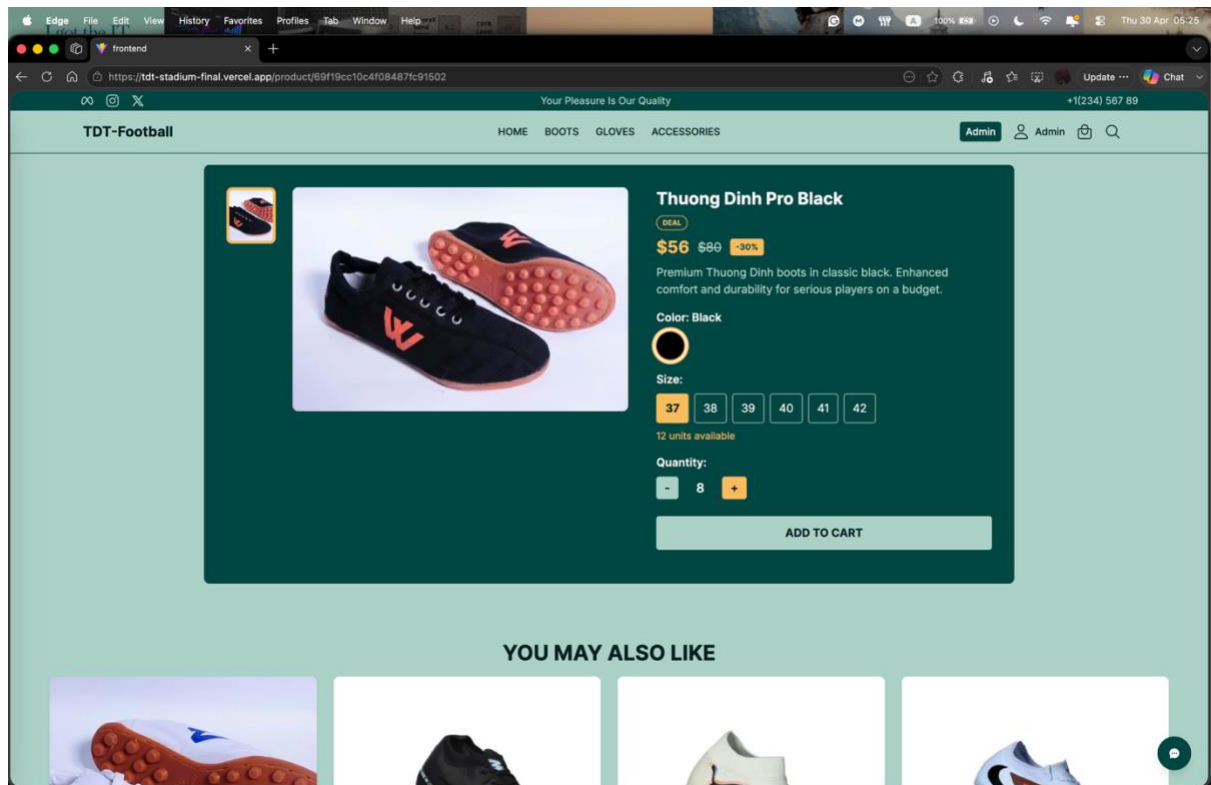
NEW BEST SELLER
ADIDAS F50 ELITE LL FG PURPLE
\$265.5
39 40 41 42 43

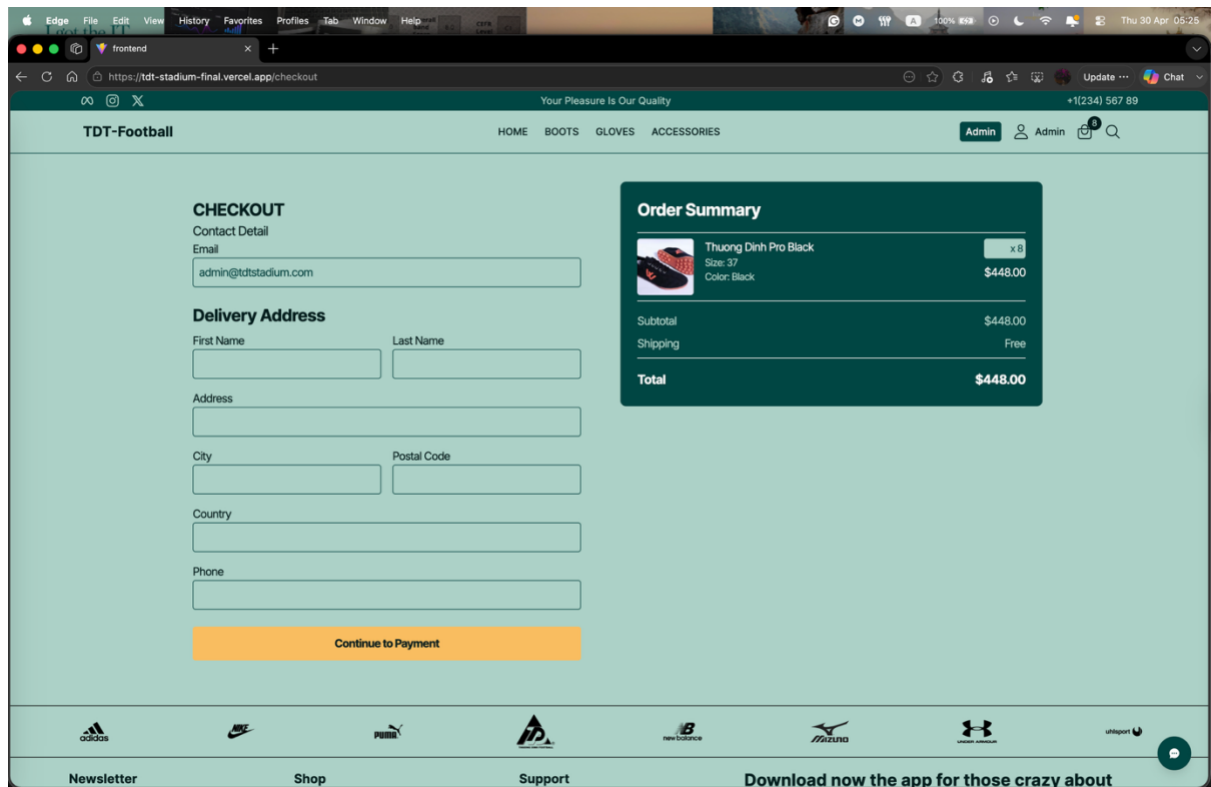
DEAL
NIKE PHANTOM 6 LOW PRO FG
\$216 ~~\$270~~

DEAL BEST SELLER
NIKE AIR ZOOM MERCURIAL VAPOR
\$237.8 ~~\$290~~

https://tdt-stadium-final.vercel.app/product/69f19cc00c4f08487c914d7







Newsletter

Shop

Support

Download now the app for those crazy about

